

UNIVERSIDADE FEDERAL DO RIO DE JANEIRO
INSTITUTO DE COMPUTAÇÃO
INSTITUTO TÉRCIO PACITTI DE APLICAÇÕES E PESQUISAS COMPUTACIONAIS
PROGRAMA DE PÓS-GRADUAÇÃO EM INFORMÁTICA (PPGI)

BRUNO LLACER TROTTI

A COMPOSITIONAL AND DIAGRAMMATIC SYNTAX FOR PARAMETRIC LINEAR
PROGRAMMING

RIO DE JANEIRO

2026

Bruno Llacer Trotti

A COMPOSITIONAL AND DIAGRAMMATIC SYNTAX FOR PARAMETRIC LINEAR
PROGRAMMING

Dissertação de Mestrado apresentada ao Programa de
Pós-Graduação em Informática, Instituto de Computação
e Instituto Tércio Pacitti de Aplicações e Pesquisas
Computacionais, Universidade Federal do Rio de Janeiro,
como requisito parcial à obtenção do título de Mestre
em Informática.

Orientador: Prof. João Paixão, D.Sc.

Rio de Janeiro
2026

Bruno Llacer Trotti

A COMPOSITIONAL AND DIAGRAMMATIC SYNTAX FOR PARAMETRIC LINEAR
PROGRAMMING

Dissertação de Mestrado apresentada ao Programa de
Pós-Graduação em Informática, Instituto de Computação
e Instituto Tércio Pacitti de Aplicações e Pesquisas
Computacionais, Universidade Federal do Rio de Janeiro,
como requisito parcial à obtenção do título de Mestre
em Informática.

Aprovado em:

de

de 2026

Prof. João Paixão, D.Sc., PPGI/UFRJ (Presidente)

Prof(a). TODO — Membro da Banca, D.Sc., Instituição

Prof(a). TODO — Membro da Banca, D.Sc., Instituição

RESUMO

Esta dissertação investiga como problemas de otimização podem ser compreendidos dentro de um arcabouço composicional e diagramático unificado, fundamentado na teoria das categorias. A motivação central é a observação de que estruturas matemáticas essenciais em otimização — relações, funções convexas, formas quadráticas, poliedros e programas lineares — podem ser interpretadas como morfismos em categorias nas quais a composição corresponde à eliminação ou minimização de variáveis internas. A partir desta perspectiva, a otimização emerge não como uma coleção de técnicas isoladas, mas como uma operação algébrica inerente à estrutura relacional. O desenvolvimento segue uma progressão incremental, partindo das relações binárias clássicas enriquecidas por quantais de Lawvere. A cada etapa é introduzido um único gerador novo e um conjunto finito de axiomas. A Álgebra Quadrática Gráfica (GQA) representa problemas de minimização da norma ℓ_2 sobre subespaços afins; a Álgebra Poliedral enriquece o arcabouço com restrições de desigualdade linear; e a Álgebra Gráfica para Programação Linear Paramétrica (GLP) constitui a contribuição central da tese, na qual um programa linear é tratado como um morfismo compositável com outros blocos de otimização. Para cada álgebra são demonstrados resultados de solidez e completude com respeito às categorias semânticas correspondentes. Um resultado central mostra que toda função convexa afim por partes admite representação como função valor de um programa linear, e que essa equivalência se manifesta diagramaticamente como uma forma normal canônica, eliminando no nível sintático a distinção entre um LP e uma função convexa. O cálculo diagramático para GLP desempenha, para a programação linear, o mesmo papel que a Álgebra Linear Gráfica desempenha para matrizes: fornece uma linguagem de formas normais, regras de reescrita e teoremas de completude dentro dos quais argumentos de otimização podem ser conduzidos simbolicamente, antes e independentemente de qualquer computação numérica.

Palavras-chave: teoria das categorias; diagramas de corda; programação linear; otimização convexa; categorias monoidais simétricas; relações ponderadas; quantais; álgebra gráfica; formas normais; completude.

ABSTRACT

This dissertation investigates how optimization problems can be understood within a unified compositional and diagrammatic framework grounded in category theory. The central observation is that key mathematical structures in optimization can be interpreted as morphisms in categories whose composition corresponds to the elimination or minimization of internal variables. From this perspective, optimization emerges not as a collection of isolated techniques but as an algebraic operation intrinsic to relational structure.

The development follows an incremental progression rooted in Lawvere quantales and weighted relations, adding one new generator and a finite set of axioms at each step. We recall the theory of Quantale relations and the Polyhedral Algebra framework as preliminaries steps to develop the novel Graphical Linear Programming (GLP), where morphisms – diagrams – are parametric linear programs. Those composes with other optimization blocks, so that the elimination of internal variables, or, in this case, the optimization itself, corresponds exactly to categorical composition. Soundness and completeness results for the previous algebras are recalled from prior works, and in the last section we also present the completeness theorem for our new language.

The diagrammatic calculus plays the same role for linear programming that Graphical Linear Algebra plays for matrices: it provides a expressible language for normal forms and rewriting rules in which arguments can be carried out symbolically, prior to and independently of numerical computation, making the main properties of the system evident in the syntax itself. The tools and results developed and proved here are this diagrammatic calculus for Linear Programming.

Keywords: category theory; string diagrams; linear programming; convex optimization; symmetric monoidal categories; weighted relations; quantales; graphical algebra; normal forms; completeness.

SUMÁRIO

1	INTRODUCTION	1
2	BACKGROUND	6
2.1	RELATIONS AND QUANTALES	6
2.2	COMPOSABILITY OF CONVEX PROBLEMS	20
3	POLYHEDRAL RELATIONS	26
3.1	SEMANTICS: THE PROP PCREL OF POLYHEDRAL CONES	26
3.2	SYNTAX: THE ALGEBRA $\text{GPA}_{\perp}$	27
3.3	THE ORDER STRUCTURE OF THE INEQUALITY GENERATOR	28
3.4	FOURIER–MOTZKIN ELIMINATION.....	31
3.5	PRESENTATION RESULTS FOR PCREL.....	38
3.6	SEMANTICS: THE PROP POLYREL OF POLYHEDRA.....	42
3.7	SYNTAX: THE ALGEBRA GPA.....	42
3.8	PRESENTATION RESULTS FOR POLYREL	43
4	THE LINEAR PROGRAMMING CATEGORY	46
4.1	SEMANTICS: THE PROP LPREL	46
4.2	SYNTAX: THE ALGEBRA GLP	48
4.3	PRESENTATION RESULTS FOR LPREL	55
5	CONCLUSION	58
	Statement on the Use of Artificial Intelligence.....	61
A	CATEGORICAL AND GRAPHICAL LINEAR ALGEBRA	65
A.1	CATEGORIES: SYSTEMS AND THEIR COMPOSITION.....	65
A.2	PARALLEL COMPOSITION AND STRING DIAGRAMS	66
A.3	PROPS AND GRAPHICAL THEORIES	67
A.4	MONOIDS, COMONIDS, AND FROBENIUS STRUCTURE	68
A.5	THE PROP OF LINEAR RELATIONS	69
A.6	GRAPHICAL LINEAR ALGEBRA.....	70
A.6.1	The equations of GLA	71
A.6.2	Notation and derived rules used in this thesis	72

1 INTRODUCTION

Mathematics is permeated by this notion of what one would call an “ideal” object. This notion of “ideal” often refers to the simplest object that meets a specific criterion, such as being the smallest element in a set, or the one through which all others factor uniquely. This particular description of “ideal” reveals an interesting interconnection between categories and optimization [Kad26]. Many of the constructions mathematics singles out as canonical are optimal in this same sense: they are the best object of a collection, with respect to an ordering or universal property carried by that collection. This thesis aims to explore this idea and provide a formal and intuitive view on such topics.

Consider for instance the familiar transitivity condition for a relation $R \subseteq X \times X$:

$$(\exists y \in X : xRy \wedge yRz) \implies xRz, \quad x, z \in X.$$

Here the premise demands the existence of an intermediate object, namely y . Nothing forces y to be unique; in fact, it is often the case that we have a set of such objects. The connection with an “ideal” object can be made explicit if we change the settings of the problem. Consider the indicator function representation of R in the sense of convex analysis [Roc70], $R : X \times X \rightarrow \{0, \infty\}$, defined by

$$R(x, y) = \begin{cases} 0, & \text{if } xRy, \\ \infty, & \text{otherwise.} \end{cases}$$

Then the above is the same as:

$$\inf_{y \in X} (R(x, y) + R(y, z)) = 0 \implies R(x, z) = 0,$$

where $\inf \emptyset = \infty$. Since the values are either zero or infinity, this is also equivalent to

$$\inf_{y \in X} (R(x, y) + R(y, z)) \geq R(x, z)$$

The resemblance is not mere coincidence: these are particular cases of a more general construction using quantales [Law73]. More generally, let Q be a unital quantale: a complete lattice equipped with an associative operation $\otimes$, with unit e , that preserves arbitrary joins in each argument. By interpreting $\exists$ and $\inf$ through the lattice join operator $\bigvee$, and $\wedge$ and $+$ through the monoidal operator $\otimes$, we obtain, for a Q -valued relation $R : X \times X \rightarrow Q$, the generalized transitivity condition

$$\bigvee_{y \in X} R(x, y) \otimes R(y, z) \leq_Q R(x, z).$$

Notice that for the cost quantale $([0, \infty], \geq, +)$ we have $a \leq_Q b \iff a \geq b$, so this replacement recovers exactly the same expression as above.

However, there is still more to it: as one may notice, the commonalities between these equations go beyond the simply visual structure; they actually share an *algebraic* structure. Explicitly, all the expressions on the left of the abstract inequality perform the very important role of *variable elimination*. The two terms share the same middle variable y , and taking the join over y produces a new expression depending only on x and z .

A join, however, already has an optimization characterization: for $A \subseteq Q$, $\bigvee A$ is the least of the upper bounds of A , the “ideal” upper bound inside the order of Q . Here the collection being aggregated is

$$A_{x,z} = \{R(x, y) \otimes R(y, z) : y \in X\}.$$

Eliminating the intermediate variable therefore computes an optimal aggregate value. The choice of Q fixes both the order used for aggregation and the operation used to combine values. Ordinary existential quantification is thus a particular case of this order-theoretic aggregation, and its indicator function makes the connection with optimization exact.

This variable elimination is precisely how composition of quantale-valued relations is defined. Given $R : X \times Y \rightarrow Q$ and $S : Y \times Z \rightarrow Q$, we have

$$(S \circ R)(x, z) := \bigvee_{y \in Y} R(x, y) \otimes S(y, z).$$

The generalized transitivity condition above is therefore simply

$$R \circ R \leq_Q R,$$

with the inequality understood pointwise. In this setting, composition is variable elimination, and the join gives it the optimization interpretation described above. Sections 2.1 and 2.2 make this precise.

Category theory gives us an abstraction to describe such operations: it explicitly encodes the notion of composition inside its own definition. Quantale-valued relations form a category under the operation above, with identity taking value e on the diagonal and $\perp$ elsewhere. More generally, composition in a category $\mathcal{C}$ is given by

$$\circ : \mathcal{C}(Y, Z) \times \mathcal{C}(X, Y) \longrightarrow \mathcal{C}(X, Z),$$

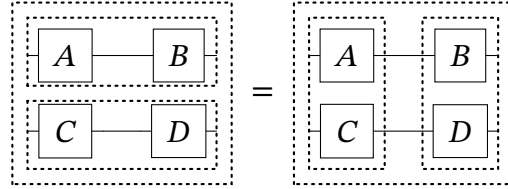
subject to associativity and identity laws. Its particular expression as a join over an intermediate variable uses the additional quantale-valued structure.

We show that, in the categories treated here, solving an optimization problem amounts to composing morphisms, and that composition carries out the variable elimination such a solution requires. Convex optimization then admits a single compositional presentation [SS24; Han+24], in which optimization is an algebraic operation rather than a technique applied from outside. That is, optimization and variable elimination are a syntactic operation, namely the composition

- of two objects A and B that share a common variable, or type.

This compositional view of optimization becomes especially apparent when analyzed through the language of string diagrams [JS91; Sel10; PZ23]. Originating as a rigorous graphical language for monoidal categories, string diagrams provide an intuitive yet mathematically precise syntax in which the sequential and parallel composition of morphisms correspond directly to vertical and horizontal juxtaposition of diagrams (e.g., representing computational processes) [JS91; Lan98]. Graphical languages are remarkably well suited to the case at hand. They make the semantics of the underlying mathematics explicit at the syntax level, in the sense that one doesn't need to go through atomic and subtle operations in order to express the core ideas of a theory, *but the system's main properties are explicit in the syntax operations themselves*.

For example, consider the following diagram. Because of the monoidal structure one may parse it by reading in two manners: composing the boxes in the same line and then stack them together, or stacking the boxes that are vertically aligned and then composing them sequentially, as the dotted boxes shows.



This denotes the fact that the composition and the monoidal product interact nicely, or, formally, that the operator $\otimes$ is actually an endofunctor, so $a; b \otimes c; d = a \otimes c; b \otimes d$. Observe how this entire property is underloaded by the syntax, so one don't even need to keep track of it but simply parse the diagram in any direction. This is what we mean when we say that the main properties are in the syntax itself, we don't get to invoke the definition or property to show the equality, but the very depicting (or drawing) embeds the property.

A fine example, that will be later explained, is one that uses the very syntax developed in this work. Consider the equivalence between these two Linear Programs

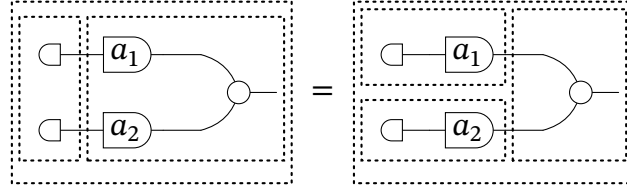
$$f(y) = \inf_{x_1, x_2} \{x_1 + x_2 \mid a_1 x_1 + a_2 x_2 = y\}$$

$$f_2(y) = \inf_{w_1, w_2} \{ \inf_{x_1} \{x_1 \mid a_1 x_1 = w_1\} + \inf_{x_2} \{x_2 \mid a_2 x_2 = w_2\} \mid w_1 + w_2 = y \}$$

These two functions are, in fact, the same. To see this, introduce the change of variables $w_1 = a_1 x_1$, $w_2 = a_2 x_2$.

Assuming $a_1, a_2 \neq 0$, each inner optimization problem has a unique feasible point, so substituting $w_i = a_i x_i$ yields $f_2(y) = \inf_{a_1 x_1 + a_2 x_2 = y} \{x_1 + x_2\} = f(y)$.

Thus, the second formulation does not change the underlying optimization problem; rather, it decomposes the original constraint through the intermediate variables w_1 and w_2 . Now look at the diagram denoting these same problems.



Once again, both problems are depicted as the very same diagram, being just a different order of parsing it. In this case all the variable changing was hidden by the syntax, and only the main semantics – the *problem itself* – is exposed.

Other areas have far off recognize the benefits of such denotation, mainly in engineering fields such as *quantum computing* [CD11] and *electronics* [BF18]. More recently, mathematicians and computer scientists have also built upon this foundation deriving a rich family of *diagrammatic* or *graphical calculi*, offering complete axiomatisations of various algebraic structures purely through equational diagram rewriting [Sel10; BSZ17]. These calculi have proved remarkably versatile, finding applications across a wide range of fields: *linear algebra* [BSZ17; Bon+19; Fre+25]; and in *optimisation and probability* [Ste+24; SS21; BDS21].

Throughout this work we explore how, step by step, one can enrich the already known theories for linear [BSZ17] algebra by adding one new generator at a time, along with a particular set of axioms, and derive even richer structures for convex problems [SS24].

The development followed a strict linear discipline, adding exactly one new generator and verifying that the resulting algebra is sound, expressive, and complete with respect to a precisely defined semantic category. Each new class of optimization problems then costs a single additional generator and a finite set of axioms. This is made precise by the normal form and completeness results of Theorem 3.3 and Theorem 4.1, which follow the same pattern already established for graphical linear and affine algebra [BSZ17; Bon+19] and for their quadratic [Ste+24] and polyhedral [BDS21] extensions.

We start by recalling how the Symmetric Monoidal Theory of Graphical Affine Algebra (**GAA**) [Bon+19] can be extended into the even more general theory of Symmetric Inequality Monoidal Theory. This is achieved by adding the inequality generator $\overline{\geq}$. Geometrically, this allows our algebra to go beyond affine spaces and start representing polyhedral figures as linear inequality constraints in optimization.

$$\{|x \in P\} = \begin{cases} 0, & Ax + b \geq 0, \\ +\infty, & \text{otherwise.} \end{cases}$$

Inspired by these observations, and parallel works in this direction [Ste+24; Law73], we make our central contribution in the final chapter: the construction and completeness of a graphical algebra for parametric linear programming. In this setting, a linear program is represented as a morphism that can be composed with other optimization blocks, leading to the idea of *composing optimization problems*. This gives a symbolic language in which the elimination of internal

variables corresponds directly to categorical composition, allowing optimization problems to be manipulated diagrammatically before computation.

In particular, every piecewise affine convex function can be represented as the value function of a linear program [Roc70], which means the Graphical Linear Programming **GLP** theory provides a syntax for a broad class of nonsmooth convex models that appear in optimization, control, economics, and machine learning. The diagrammatic syntax therefore erases the distinction between expressing a linear program as an optimization problem and expressing it as a piecewise affine convex function. As the example above, we have only one syntax to denote one object, instead of two distinct ways to syntactically depict it.

Thesis overview In Section 2.1 we set up the algebraic and categorical setting on which the rest of the work is built, introducing quantales and relations and the interchangeable role they play in a categorical view. Next, in Section 2.2, we apply this theory to convex optimization [SS24; Kad26], showing how several of its central constructs fall within the same categorical framework.

Chapter 3 presents the already-known graphical theory of Polyhedral Algebra [BDS21], where some of the known proofs are recalled and refined.

Finally, in Chapter 4, we present the central contribution of this thesis: an entirely new Graphical Algebra, **GLP**, dedicated to expressing parametric Linear Programs. Building on the polyhedral generators of Chapter 3, we introduce a single new generator for the linear objective function and derive a complete axiomatization for right-hand side parametric linear programming, proving that **GLP** presents the prop **LPRel**. This work represents a first step toward a complete diagrammatic calculus for convex optimization.

2 BACKGROUND

2.1 RELATIONS AND QUANTALES

In this section we explore the theory of relations. On set-theoretical foundations, relations are among the most primitive notions available, and yet they already support remarkably powerful abstractions: many other structures, functions among them, are derived from relations alone. Later we shall see that relations sit comfortably within the language of category theory, which in turn offers a natural vocabulary for describing them.

We begin with the most basic kind of relation:

Definition 2.1 (Binary Relations). *Let X and Y be sets. A binary relation R from X to Y is a subset*

$$R \subseteq X \times Y.$$

Equivalently, it can be identified with its characteristic function

$$\chi_R : X \times Y \rightarrow \{0, +\infty\},$$

defined by

$$\chi_R(x, y) = \begin{cases} 0 & \text{if } (x, y) \in R, \\ +\infty & \text{otherwise.} \end{cases}$$

A binary relation specifies whether x is related to y . Observe that one can think of a relation in two different ways that will prove useful later. First, as a set of pairs, where every relation is just a pair $(x, y) \in R$, or as a map that has x, y as inputs and outputs 0 if they are related, and $+\infty$ if they are not the choice of ∞ is arbitrary and will be justified later. Normal choices here are actually $\{true, false\}$ or $\{0, 1\}$.

This is a general construct as the sets X and Y can be anything. We can, for instance, ask for further structures on such sets and obtain different kinds of relations.

Definition 2.2 (Linear/Affine Relations). *Let X and Y be vector spaces. A linear relation from X to Y is a linear subspace*

$$R \subseteq X \times Y.$$

An affine relation is an affine subspace of $X \times Y$.

Equivalently, a linear relation can be described as the graph of a possibly multivalued linear operator, i.e.,

$$R(x) := \{y \in Y \mid (x, y) \in R\}.$$

or, its characteristic function defined by

$$\chi_R : X \times Y \rightarrow \{0, +\infty\},$$

$$\chi_R(x, y) = \begin{cases} 0 & \text{if } (x, y) \in R, \\ +\infty & \text{otherwise.} \end{cases}$$

Thus, a linear/affine relation is a binary relation where the set R is a subspace

Notice, however, that we have been restricting the relations between x and y to be a case of "they are, or they are not related". In fact, we might be interested in adding degrees to such a relation, so that we could say that x and y are "more" or "less" related. This leads us to the well-known notion of weighted relations [Law73].

Definition 2.3 (Weighted Relations). *Let X and Y be any two sets and let $\overline{\mathbb{R}}^+ := [0, +\infty]$. A weighted relation from X to Y is a set*

$$R \subseteq X \times Y \times \overline{\mathbb{R}}^+$$

such that for every $(x, y) \in X \times Y$ there exists exactly one $r \in \overline{\mathbb{R}}^+$ satisfying

$$(x, y, r) \in R.$$

Equivalently, we may define the characteristic function

$$\chi_R : X \times Y \rightarrow \overline{\mathbb{R}}^+$$

by

$$\chi_R(x, y) = r \quad \text{if and only if } (x, y, r) \in R.$$

and the codomain is not restricted to a binary set anymore. In fact, one could choose any other set instead of the $\overline{\mathbb{R}}^+$ to be the codomain of the characteristic function as long as it satisfy some properties. In fact, the whole structure of a relation is further generalized by the notion of a quantale.

Definition 2.4 (Quantale). *A quantale is a structure $(Q, \leq, \otimes, e)$ such that:*

- $(Q, \leq)$ is a complete lattice, i.e., every subset $A \subseteq Q$ admits a join $\bigvee A$ and a meet $\bigwedge A$;
- $(Q, \otimes, e)$ is a monoid;
- Multiplication distributes over arbitrary join in each variable:

$$a \otimes \left(\bigvee_{i \in I} b_i \right) = \bigvee_{i \in I} (a \otimes b_i), \quad \left(\bigvee_{i \in I} a_i \right) \otimes b = \bigvee_{i \in I} (a_i \otimes b)$$

for all families $\{a_i\}_{i \in I}, \{b_i\}_{i \in I} \subseteq Q$.

If $\otimes$ is commutative, the quantale is called commutative. If $e = \top$ (the top element of the lattice), it is called integral.

Each of these three axioms earns its place. The complete lattice gives meaning to the join $\vee$, the operation that eliminated the intermediate variable y in the transitivity example of the introduction. The monoid $(Q, \otimes, e)$ plays the role of combining evidence along a chain of relations, with e the trivial combination that changes nothing. Distributivity is what lets us freely interchange this join and this product, precisely the property that allows composition and variable elimination to be reduced to the same operation, rather than two operations that merely happen to look alike.

Given a quantale Q , we can define the notion of Q -valued relation [Coe+18].

Definition 2.5 (Generalized Relations). *A generalized relation (or Q -valued relation) from X to Y , where Q is a quantale, is a set*

$$R \subseteq X \times Y \times Q$$

such that for every $(x, y) \in X \times Y$ there exists exactly one $q \in Q$ satisfying

$$(x, y, q) \in R.$$

Its characteristic function is

$$\chi_R : X \times Y \rightarrow Q$$

defined by

$$\chi_R(x, y) = q \quad \text{if and only if } (x, y, q) \in R.$$

Now we are ready to build our previous relations as particular cases of this more general framework.

Example 2.1. *The notion of a Q -valued relation recovers familiar concepts for particular choices of quantale.*

(1) Truth quantale. *Let $Q = \{0, 1\}$ with the usual order $0 \leq 1$, join given by $\vee$, and monoidal product $\otimes = \wedge$ with unit $e = 1$. This is the (Boolean) truth quantale.*

A Q -valued relation is then a function

$$R : X \times Y \rightarrow \{0, 1\}.$$

Such a function is precisely the characteristic function of an ordinary binary relation

$$\tilde{R} \subseteq X \times Y, \quad (x, y) \in \tilde{R} \iff R(x, y) = 1.$$

Thus Q -valued relations coincide with classical binary relations.

This also settles the indicator convention adopted in Definition 2.1, where relations were valued in $\{0, +\infty\}$ rather than $\{0, 1\}$. The two are the same quantale: the map $1 \mapsto 0, 0 \mapsto +\infty$ sends $\vee$ to $\min$, $\wedge$ to $+$, and the unit 1 to the unit 0 , hence is an isomorphism of the truth quantale

onto the sub-quantale $\{0, +\infty\}$ of the Lawvere quantale below. We work with $\{0, +\infty\}$ throughout, so that Boolean relations sit inside the cost-valued ones without any change of convention.

(2) **Lawvere quantale.** Let $Q = \overline{\mathbb{R}}^+ = [0, +\infty]$ ordered by the reverse of the usual order (so that $a \leq b$ iff $a \geq b$ in the usual order), with monoidal product given by addition

$$a \otimes b := a + b,$$

and unit $e = 0$. Joins are then given by infima in the usual order.

This structure is the Lawvere quantale. A Q -valued relation

$$R : X \times Y \rightarrow \overline{\mathbb{R}}^+$$

is precisely a weighted relation assigning a (possibly infinite) cost to each pair (x, y) .

Hence generalized relations unify ordinary binary relations and weighted relations as special cases corresponding to particular quantales.

Note that linear and affine relations are not recovered by this construction, since they demand extra structure on the sets X and Y , specifically that R be a linear or affine subspace of $X \times Y$. They are, nonetheless, fully embedded in the framework of binary relations.

Within this framework, we propose an extension of the Lawvere Quantale and weighted relations called the Extended-Lawvere Quantale and the Cost/Reward Relations, respectively. To our knowledge, this definition does not appear elsewhere in the literature.

Definition 2.6 (Optimization/Extended Lawvere Quantale). *Let*

$$\overline{\mathbb{R}} := \mathbb{R} \cup \{-\infty, +\infty\}.$$

The optimization quantale is the structure

$$(\overline{\mathbb{R}}, \geq, \otimes, e)$$

defined as follows:

- *The order $\geq$ is the usual order on the extended real line. With this order, arbitrary joins are given by the usual infima:*

$$\bigvee A = \inf A.$$

- *The monoidal product is addition:*

$$a \otimes b := a + b,$$

where addition is extended to $\overline{\mathbb{R}}$ by $a + (+\infty) = +\infty$ for every a , and $a + (-\infty) = -\infty$

for every $a \neq +\infty$. In particular we adopt the convention

$$(+\infty) + (-\infty) := +\infty,$$

so that $+\infty$ is absorbing. This is Moreau's inf-addition [Roc70], and it is the only extension under which the structure below is a quantale.

- The unit element is

$$e := 0.$$

With these operations, $(\overline{\mathbb{R}}, \geq, +, 0)$ is a commutative quantale. Joins are ordinary infima, and addition distributes over arbitrary joins.

The convention above is not a matter of taste. Without it, $+$ is only partially defined on $\overline{\mathbb{R}}$, so $(\overline{\mathbb{R}}, +, 0)$ fails to be a monoid and the quantale axioms cannot even be stated. The next proposition checks that inf-addition repairs this.

Proposition 2.1. $(\overline{\mathbb{R}}, \geq, +, 0)$ with inf-addition is a commutative quantale.

Proof. Since $\overline{\mathbb{R}}$ under the reversed order is a complete lattice with $\bigvee A = \inf A$, $\perp = \bigvee \emptyset = +\infty$ and $\top = -\infty$, only the monoid structure and distributivity need checking.

(1) Monoid. Inf-addition is commutative by construction and total, since the one ambiguous sum is now assigned the value $+\infty$. For associativity, note that $a + b = +\infty$ exactly when $a = +\infty$ or $b = +\infty$, and $a + b = -\infty$ exactly when neither is $+\infty$ and at least one is $-\infty$. Both conditions are symmetric in the summands and depend only on the multiset $\{a, b\}$, so $(a + b) + c$ and $a + (b + c)$ agree on every triple: each equals $+\infty$ if some argument is $+\infty$, else $-\infty$ if some argument is $-\infty$, else the ordinary finite sum. The unit is 0.

(2) Distributivity. Let $a \in \overline{\mathbb{R}}$ and $\{b_i\}_{i \in I}$ be any family. We must show $a + \inf_i b_i = \inf_i (a + b_i)$, where both sides are $+\infty$ when $I = \emptyset$.

If $a = +\infty$, both sides are $+\infty$ by absorption. If a is finite, the identity is the standard translation invariance of the infimum. If $a = -\infty$ and every $b_i = +\infty$, then $\inf_i b_i = +\infty$ and both sides are $+\infty$; if instead some $b_{i_0} < +\infty$, then $\inf_i b_i < +\infty$, so the left side is $-\infty$, and the right side is $-\infty$ as well because $a + b_{i_0} = -\infty$. Commutativity gives distributivity in the other variable. $\square$

Definition 2.7 (Optimization (Cost/Reward) Relations). Let X and Y be sets. An optimization relation from X to Y is an $\overline{\mathbb{R}}$ -valued relation, i.e., a function

$$R : X \times Y \rightarrow \overline{\mathbb{R}}.$$

Equivalently, it is a subset

$$R \subseteq X \times Y \times \overline{\mathbb{R}}$$

such that for every $(x, y) \in X \times Y$ there exists exactly one $r \in \overline{\mathbb{R}}$ satisfying

$$(x, y, r) \in R.$$

The value $R(x, y)$ is interpreted as the objective value (cost or reward) associated to the pair (x, y) , with $+\infty$ and $-\infty$ representing extremal infeasibility or unboundedness.

Observe how this is essentially the weighted relations extended along the negative values. The importance of it is that we want to be able to express optimization problems that might be unfeasible ($+\infty$) or unbounded ($-\infty$), so we need to add both the top and bottom element.

All of this admits a suitable and natural categorical construction. Specifically, from any quantale Q one can not only define Q -relations, but also construct an entire category of them. This category is the natural setting for discussing the central concept of this framework: the composition of relations.

The following lemma shows how to build such a category and establishes that this category also has a very important monoidal structure behind it. Particularly, the category is a compact closed symmetric monoidal category [Lan98]:

Proposition 2.2 (Category $\mathbf{Rel}(Q)$). *For a quantale Q , the Q -relations form a category $\mathbf{Rel}(Q)$ with composition and identities*

$$(S \circ R)(a, c) = \bigvee_b R(a, b) \otimes S(b, c) \quad 1_A(a, b) = \bigvee \{k \mid a = b\}$$

If Q is a commutative quantale, then $\mathbf{Rel}(Q)$ carries a symmetric monoidal structure, with the categorical monoidal product $\otimes_{\mathcal{C}}$ on objects given by the Cartesian product of sets and the action on relations given for $R : A \rightarrow C$ and $S : B \rightarrow D$ by

$$(R \otimes_{\mathcal{C}} S)(a, b, c, d) = R(a, c) \otimes S(b, d)$$

The singleton set is the monoidal unit, and the category is compact closed w.r.t the monoidal structure

Proof. We verify the categorical structure.

(1) Well-defined composition. Let $R : A \rightarrow B$ and $S : B \rightarrow C$ be Q -relations. For each $(a, c) \in A \times C$, the set

$$\{R(a, b) \otimes S(b, c) \mid b \in B\} \subseteq Q$$

admits a supremum since Q is a complete lattice. Hence $(S \circ R)(a, c)$ is well-defined as an element of Q .

(2) Associativity. Let $R : A \rightarrow B$, $S : B \rightarrow C$, and $T : C \rightarrow D$. For $a \in A$, $d \in D$,

$$((T \circ S) \circ R)(a, d) = \bigvee_c \left(\bigvee_b R(a, b) \otimes S(b, c) \right) \otimes T(c, d).$$

Since $\otimes$ distributes over arbitrary suprema,

$$= \bigvee_{b,c} R(a, b) \otimes S(b, c) \otimes T(c, d).$$

Similarly,

$$(T \circ (S \circ R))(a, d) = \bigvee_b R(a, b) \otimes \left(\bigvee_c S(b, c) \otimes T(c, d) \right) = \bigvee_{b,c} R(a, b) \otimes S(b, c) \otimes T(c, d).$$

Thus composition is associative.

(3) Identities. For each set A , define

$$1_A(a, b) = \begin{cases} e & \text{if } a = b, \\ \perp & \text{otherwise,} \end{cases}$$

where $\perp = \bigvee \emptyset$ is the bottom element of Q .

Then for $R : A \rightarrow B$,

$$(R \circ 1_A)(a, b) = \bigvee_{a'} 1_A(a, a') \otimes R(a', b).$$

All terms vanish except when $a' = a$, so

$$= e \otimes R(a, b) = R(a, b).$$

Similarly, $1_B \circ R = R$. Hence identities exist.

(4) Monoidal structure (commutative case). Assume Q is commutative. Define the tensor on objects by Cartesian product:

$$A \otimes_{\mathcal{C}} B := A \times B.$$

For relations $R : A \rightarrow C$ and $S : B \rightarrow D$, define

$$(R \otimes_{\mathcal{C}} S)((a, b), (c, d)) = R(a, c) \otimes S(b, d).$$

Functoriality follows from associativity and commutativity of $\otimes$:

$$(S_1 \otimes S_2) \circ (R_1 \otimes R_2) = (S_1 \circ R_1) \otimes (S_2 \circ R_2).$$

The singleton set 1 is a unit since $A \times 1 \cong A$.

Symmetry is induced by the swap map

$$\sigma_{A,B}((a, b), (b', a')) = \begin{cases} e & \text{if } a = a', b = b', \\ \perp & \text{otherwise.} \end{cases}$$

(5) Compact closure. For each set A , define unit and counit relations

$$\eta : 1 \rightarrow A \times A, \quad \epsilon : A \times A \rightarrow 1$$

by

$$\eta(*, (a, a')) = \begin{cases} e & a = a', \\ \perp & \text{otherwise,} \end{cases} \quad \epsilon((a, a'), *) = \begin{cases} e & a = a', \\ \perp & \text{otherwise.} \end{cases}$$

A direct computation using the definition of composition shows that the triangle identities hold:

$$(1_A \otimes \epsilon) \circ (\eta \otimes 1_A) = 1_A, \quad (\epsilon \otimes 1_A) \circ (1_A \otimes \eta) = 1_A.$$

Thus every object is self-dual, and $\mathbf{Rel}(Q)$ is compact closed. $\square$

A curious fact about relations is that they actually encode the notion of matrices and matrix multiplication. This serves as motivation for what will later become the diagrammatical theory of linear algebra. The following remark expands on it showing that every morphism in $\mathbf{Rel}(Q)$ corresponds to a matrix with entries in Q , and that composition is exactly matrix multiplication.

Remark 2.1. A Q -valued relation $R : X \times Y \rightarrow Q$ is nothing but a Q -valued matrix indexed by $X \times Y$: one simply reads off the entry at position (x, y) as $\chi_R(x, y) \in Q$. That is, a generalized relation is precisely an element of $Q^{X \times Y}$.

When X and Y are finite sets, say $|X| = m$ and $|Y| = n$, this becomes completely explicit: a Q -valued relation is exactly an $m \times n$ matrix

$$M_R = [\chi_R(x_i, y_j)]_{\substack{1 \leq i \leq m \\ 1 \leq j \leq n}} \in Q^{m \times n},$$

whose (i, j) -entry records the Q -value assigned to the pair (x_i, y_j) .

Moreover, composition of Q -valued relations corresponds precisely to matrix multiplication in Q . Given a second Q -valued relation $S : Y \times Z \rightarrow Q$, their composite $S \circ R : X \times Z \rightarrow Q$ is defined by

$$\chi_{S \circ R}(x, z) = \bigvee_{y \in Y} \chi_R(x, y) \otimes \chi_S(y, z),$$

which, in matrix notation, reads

$$M_{S \circ R} = M_R \otimes M_S, \quad (M_R \otimes M_S)_{ij} = \bigvee_{k=1}^n (M_R)_{ik} \otimes (M_S)_{kj}.$$

This is exactly the usual row-times-column matrix product, but with ordinary addition replaced by the join $\vee$ and ordinary multiplication replaced by the monoidal product $\otimes$ of Q . The two axioms of a quantale — that $\otimes$ distributes over arbitrary joins — are precisely what guarantees that this multiplication is associative and well-defined, so that Q -valued matrices form a category under this composition.

To illustrate these new concepts, we look at some of our previous relations now built as categories.

Example 2.2 (Category of Boolean relations). *Let*

$$Q = (\{0, 1\}, \leq, \otimes, e)$$

be the Boolean quantale where:

- $0 \leq 1$;
- $\otimes = \wedge$ (logical conjunction);
- $e = 1$;
- joins are given by $\vee$.

A Q -valued relation $R : A \rightarrow B$ is a function

$$R : A \times B \rightarrow \{0, 1\},$$

which is precisely the characteristic function of a subset

$$\tilde{R} \subseteq A \times B.$$

The composition formula in $\mathbf{Rel}(Q)$ becomes

$$(S \circ R)(a, c) = \bigvee_b R(a, b) \wedge S(b, c).$$

Since joins are logical disjunction, this reduces to

$$(S \circ R)(a, c) = 1 \iff \exists b \in B \text{ such that } R(a, b) = 1 \text{ and } S(b, c) = 1.$$

Thus composition coincides exactly with the usual composition of binary relations.

Since the Boolean quantale is commutative, $\mathbf{Rel}(Q)$ carries a symmetric monoidal structure:

- *On objects:*

$$A \otimes B := A \times B.$$

- On morphisms:

$$(R \otimes S)((a, b), (c, d)) = R(a, c) \wedge S(b, d).$$

The singleton set is the monoidal unit. With this structure, $\mathbf{Rel}(Q)$ is precisely the classical symmetric monoidal compact closed category $\mathbf{Rel}$.

Example 2.3 (Category of weighted relations). Let

$$Q = (\mathbb{R} \cup \{+\infty\}, \leq_Q, \otimes, e)$$

be the Lawvere quantale where:

- $a \leq_Q b \iff a \geq b$ in the usual order;
- $\otimes = +$;
- $e = 0$;
- joins are given by infima in the usual order.

A Q -valued relation $R : A \rightarrow B$ is a function

$$R : A \times B \rightarrow \mathbb{R} \cup \{+\infty\},$$

assigning a cost (or weight) to each pair (a, b) .

The composition formula becomes

$$(S \circ R)(a, c) = \inf_{b \in B} (R(a, b) + S(b, c)).$$

Thus composition is given by infimal convolution (or shortest-path composition).

Since addition is commutative, the Lawvere quantale is commutative, and $\mathbf{Rel}(Q)$ carries a symmetric monoidal structure:

- On objects:

$$A \otimes B := A \times B.$$

- On morphisms:

$$(R \otimes S)((a, b), (c, d)) = R(a, c) + S(b, d).$$

The singleton set is the monoidal unit. With this tensor product, $\mathbf{Rel}(Q)$ is a symmetric monoidal compact closed category of weighted relations.

In particular:

- If A, B, C are finite sets, relations may be viewed as matrices with entries in $\mathbb{R} \cup \{+\infty\}$.

- *Composition is matrix multiplication in the $(\min, +)$ -semiring.*

Hence $\mathbf{Rel}(Q)$ is the category of weighted relations, which underlies dynamic programming, tropical algebra, and shortest-path problems.

Those examples illustrate how naturally the quantale structure supports categorical constructions: composition of Q -valued relations corresponds to matrix multiplication driven by $\otimes$, and the tensor product of relations is governed by the join $\vee$, both operations coming directly from the quantale axioms.

The resulting categories are symmetric monoidal, and it is precisely this monoidal backbone that will support the algebraic structure we aim to build. A key piece of that structure is a *Frobenius monoid* on each object: a monoid (μ, η) and a comonoid (δ, ϵ) on the same object, satisfying the Frobenius law

$$(\mathrm{id} \otimes \mu) \circ (\delta \otimes \mathrm{id}) = \delta \circ \mu = (\mu \otimes \mathrm{id}) \circ (\mathrm{id} \otimes \delta).$$

A symmetric monoidal category in which every object carries such a structure, compatibly with the tensor product, is called a *hypergraph category* [FS19].

Hypergraph categories are related to, but strictly more general than, compact closed categories [Sel10]. In a compact closed category every object A has a dual A^* with unit $\eta_A : I \rightarrow A^* \otimes A$ and counit $\epsilon_A : A \otimes A^* \rightarrow I$ satisfying the snake equations; a Frobenius structure does not require the existence of such a dual. Conversely, if a compact closed category is such that every object is self-dual ($A \cong A^*$) and the induced cups and caps satisfy the Frobenius law, then it is a hypergraph category. Thus compact closed categories with self-dual objects form a special class of hypergraph categories, and it is the later, more general, notion that we shall adopt as our ambient framework.

Remark 2.2. *If Q is commutative, then $\mathbf{Rel}(Q)$ is a hypergraph category and admits a commutative Frobenius structure on every object. Diagrammatically, we denote the additive and co-additive structure as*

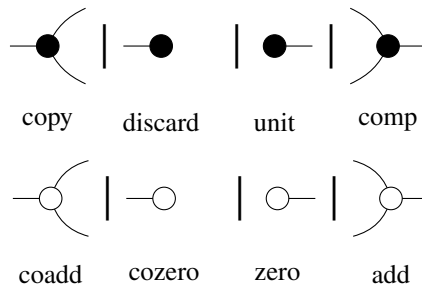
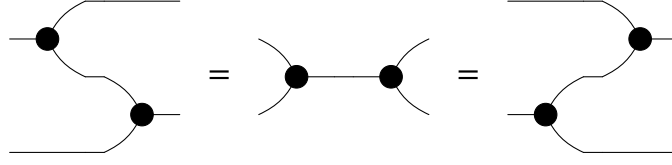


Figure 2.1 – Frobenius additive(black) and co-additive(white) structure

Subject to the Frobenius Law (both colors)



Proof. Just a check that all laws do hold for any $\mathbf{Rel}(Q)$ category □

These constructions provide exactly the semantic foundation for the algebra we aim to build. In the same way that semantics is understood in linguistics and programming language theory [Win93], the semantics of a formal system assigns meaning to its expressions. More precisely, while syntax consists of the tokens and rules for forming and combining expressions, semantics is the interpretation assigned to each such expression — in our case, an optimization problem living in $\mathbf{Rel}(Q)$.

As outlined in the introduction, the central goal of this work is to show that optimization emerges naturally from algebraic and categorical structure. The key mechanism enabling this is the infimal composition present in the definition of the Costs/Rewards category: composing two relations automatically performs variable elimination, which is precisely the fundamental operation underlying optimization.

To make this precise, we need both a semantics — provided by the categorical framework developed above — and a syntax: a rigorous, compositional language for expressing optimization problems symbolically. The map between those is structure-preserving map, a functor, from the syntactic layer into the semantic one.

The syntactic layer will be formalized as a Symmetric Monoidal Theory (SMT) [BSZ17], which is a presentation of a category by generators and equations, whose terms respect the compositional structure of a symmetric monoidal category. This is the standard tool for building graphical languages in the style of Graphical Linear Algebra, and it is the framework within which our completeness results will be stated and proved.

Definition 2.8 (Symmetric Monoidal Theory). *A SMT is a pair (Σ, E) , such that, Σ is a signature of operators o that has arity m and coarity n*

Σ -terms are defined inductively as the smallest collection of typed expressions $t : m \rightarrow n$ such that:

1. (Generators) *If $o \in \Sigma$ with $o : m \rightarrow n$, then o is a Σ -term of type $m \rightarrow n$.*
2. (Identity) *For every $n \in \mathbb{N}$, there is a term*

$$\text{id}_n : n \rightarrow n.$$

3. (Composition) *If*

$$a : m \rightarrow n \quad \text{and} \quad b : n \rightarrow k$$

are Σ -terms, then

$$a; b : m \rightarrow k$$

is a Σ -term.

4. (Tensor product) If

$$a : m_1 \rightarrow n_1 \quad \text{and} \quad b : m_2 \rightarrow n_2$$

are Σ -terms, then

$$a \otimes b : m_1 + m_2 \rightarrow n_1 + n_2$$

is a Σ -term.

5. (Symmetry) For all $m, n \in \mathbb{N}$, there is a term

$$\sigma_{m,n} : m + n \rightarrow n + m.$$

and E be a set of equations over the Σ – terms.

Often, we will think of elements $o : m \rightarrow n \in \Sigma$ as generators and represent them as string diagrams. Being the Σ – terms the results of composing those generators horizontally and vertically inductively. The set E as a collection of axiomatic equations over these string diagrams.

The categorical counterpart of these terms is the free category generated by the SMT.

Definition 2.9 (The Free category of SMT). *The $\text{FreeP}_{(\Sigma, E)}$ is the category freely generated by (Σ, E) where the objects are natural numbers, morphisms are Σ – terms quotiented by E , with composition and tensor product inherited from the symmetric monoidal structure. $\text{FreeP}_{(\Sigma, E)}$ is in fact a PROP ¹.*

The $\text{FreeP}_{(\Sigma, E)}$, also denoted as $P\Sigma_{/E}$,² will be our **syntax** responsible for expressing our underlying semantics. For it to be considered a "useful" syntax we expect that it satisfies some properties, particularly we want it to be complete, sound and expressive.

Without those, our algebra would be empty since we would have no assurances that it actually says anything meaningful about we desire to say.

Definition 2.10. *We will say that a SMT axiomatizes or represents a specific category, usually called the semantic category, when the following diagram commutes:*

¹For the definition of a prop see [Lan98]

²During the work we will use FreeP_S and S , for whatever syntax S may be, interchangeably.

$$\begin{array}{ccc}
P\Sigma_{/E} = \text{Free}P_{(\Sigma, E)} & \xrightarrow{\cong} & \text{Im}(\llbracket \cdot \rrbracket) \\
\uparrow q & \nearrow s & \downarrow i \\
\text{Syn} = P\Sigma & \xrightarrow{\llbracket \cdot \rrbracket} & \text{Sem}
\end{array}$$

where: $P\Sigma$ is the category where objects are natural numbers and morphisms are Σ – terms from $n \rightarrow m$; $P\Sigma_{/E} = \text{Free}P_{(\Sigma, E)}$ is the $P\Sigma$ category with the morphisms quotiented by E and there is then a prop morphism $q : P\Sigma \rightarrow P\Sigma_{/E}$ witnessing this quotient; **Sem** is our semantic category (the Cost/Reward relations for example); and s is a full and faithful prop morphism. Another way to say the same is to demand two things out of the SMT, namely:

- *Soundness:* We say that $P\Sigma_{/E}$ is sound if $c \stackrel{E}{=} d^3$ implies $\llbracket c \rrbracket = \llbracket d \rrbracket$.
- *Completeness:* We say that $P\Sigma_{/E}$ is complete if $\llbracket c \rrbracket = \llbracket d \rrbracket$ implies $c \stackrel{E}{=} d$
- *Expressivity:* We say that $P\Sigma_{/E}$ is expressive if for every morphism c in **Sem** there exists $d \in P\Sigma_{/E}$ such that $\llbracket d \rrbracket = c$

Those three properties capture the idea that everything we can prove in the syntax, is a holds in the semantics (soundness), everything that holds in the semantics is provable in the syntax (completeness) and that every morphism in the semantics has a syntactic representative (expressivity). Moreover, these properties denote equivalent characteristics about the interpretation map, both categorically and functionally. Below we depict a small table defining the equivalence between the desired properties and their equivalent statement:

Table 2.1 – Equivalence of properties for the interpretation map $\llbracket - \rrbracket : P\Sigma_{/E} \rightarrow \text{Sem}$

Set theory	Category theory	Logic (SMT)
Total	Functor	Soundness: $c \stackrel{E}{=} d \Rightarrow \llbracket c \rrbracket = \llbracket d \rrbracket$
Deterministic	Well-defined map	(Functionality of the interpretation)
Injective	Faithful	Completeness: $\llbracket c \rrbracket = \llbracket d \rrbracket \Rightarrow c \stackrel{E}{=} d$
Surjective	Full	Expressivity: $\forall f \in \text{Sem}, \exists d \in P\Sigma_{/E} \text{ s.t. } \llbracket d \rrbracket = f$

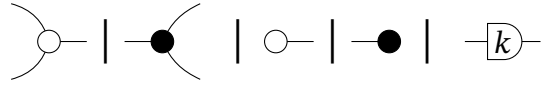
³ $\stackrel{E}{=}$ means that this equality is provable by the equations in E

Now we introduce two of the foundational theories for Graphical Algebra, that will serve as a foundation for the following sections, namely, Graphical Linear Algebra [PRS22; BSZ17] and Graphical Affine Algebra [Bon+19], we omit their definitions and proofs and refer to the original sources.

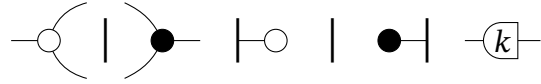
However, we do provide some notation on how to talk about those during the course of the work. Notice that, their semantics was already introduced when we talked about Linear/Affine relations.

Definition 2.11. We will call **GLA** the presentation of the Graphical Linear Algebra SMT, and, in the same manner, **GAA** the SMT of Graphical Affine Algebra.

Definition 2.12. We also denote with $\overrightarrow{(\cdot)}/\overleftarrow{(\cdot)}$ when we want to talk about the algebra generated without the Frobenius structure. For example $\overrightarrow{\text{GLA}}$ is the algebra generated by



without their symmetric



and the exactly opposite we denote as $\overleftarrow{\text{GLA}}$.

2.2 COMPOSABILITY OF CONVEX PROBLEMS

Having established the necessary categorical and algebraic tools, we proceed to their application in convex analysis. In this section we introduce an initial categorical framework for convex optimization [SS24], showing how the fundamental operations of convex analysis — infimal convolution, conjugation, and variable elimination — arise naturally as categorical constructions. This gives the first concrete evidence that optimization is not merely a collection of isolated techniques, but a coherent algebraic theory with deep compositional structure.

The progression of the section is as follows. We begin with the general category of bifunctions, which serves as the broadest setting, and progressively restrict to convex bifunctions, where the analytical structure is richest. These results lay the groundwork for the graphical theories developed in the following sections: Polyhedral Algebra and Graphical Linear Programming will both arise as full subcategories of $\mathbf{CxBiFn}$, each corresponding to a class of bifunctions with additional algebraic structure.

Later, we will see the particular applications of such definitions and results. For now, however, we begin by defining the broadest category in the hierarchy, which will generalize most of the later work: the category of bifunctions.

Definition 2.13 (Bifunction Category). The category **BiFunc** is defined as follows:

- *Objects are sets.*
- *A morphism $F : A \rightarrow B$ is a function*

$$F : A \times B \rightarrow \overline{\mathbb{R}}.$$

- *Given $F : A \rightarrow B$ and $G : B \rightarrow C$, their composition $G \circ F : A \rightarrow C$ is defined pointwise by*

$$(G \circ F)(a, c) := \inf_{b \in B} (F(a, b) + G(b, c)).$$

- *The identity morphism on an object A is the function*

$$1_A : A \times A \rightarrow \overline{\mathbb{R}}$$

given by

$$1_A(a, a') = \begin{cases} 0 & \text{if } a = a', \\ +\infty & \text{otherwise.} \end{cases}$$

If we equip **BiFunc** with a monoidal structure:

- *The tensor product on objects is the Cartesian product*

$$A \otimes B := A \times B.$$

- *For morphisms $F : A \rightarrow C$ and $G : B \rightarrow D$, their tensor product*

$$F \otimes G : A \times B \rightarrow C \times D$$

is defined by

$$(F \otimes G)((a, b), (c, d)) := F(a, c) + G(b, d).$$

The singleton set serves as the monoidal unit.

Remark 2.3. The category **BiFunc** is isomorphic to the category **RelQ** for a suitable choice of Q .

Consider the quantale

$$Q = (\overline{\mathbb{R}}, \leq_Q, \otimes, e)$$

where:

- $\overline{\mathbb{R}} = \mathbb{R} \cup \{-\infty, +\infty\};$

- the order is reversed with respect to the usual order,

$$a \leq_Q b \iff a \geq b \quad (\text{usual order});$$

- the monoidal product is addition,

$$a \otimes b := a + b;$$

- the unit is $e = 0$.

In this quantale, arbitrary joins are given by infima in the usual order:

$$\bigvee_Q A = \inf A.$$

Now a Q -valued relation $R : A \rightarrow B$ is precisely a function

$$R : A \times B \rightarrow \overline{\mathbb{R}},$$

i.e. a bifunction. Moreover, the composition formula in $\mathbf{Rel}(Q)$ becomes

$$(S \circ R)(a, c) = \bigvee_b R(a, b) \otimes S(b, c) = \inf_b (R(a, b) + S(b, c)),$$

which is exactly the composition law in $\mathbf{BiFunc}$.

Therefore, $\mathbf{BiFunc}$ is (strictly) the same category as $\mathbf{Rel}(Q)$ for the Lawvere-type quantale $(\overline{\mathbb{R}}, \geq, +, 0)$.

Bifunctions are fundamental objects in convex analysis [Roc70]. They provide a unified language for expressing objective functions, feasibility constraints, and duality simultaneously. The following proposition confirms that the framework is conservative: classical set-theoretic relations embed faithfully into this richer setting.

Proposition 2.3. *The category of Binary Relations, i.e. the Boolean-relation, is embedded as a full subcategory of $\mathbf{BiFunc}$.*

Proof. Just consider the functor $F : \mathbf{Rel}(B) \rightarrow \mathbf{BiFunc}$ that takes every morphism $R : X \rightarrow Y$, which is a subset of $X \times Y$, and maps it into the characteristic function $\chi_R : X \times Y \rightarrow \{0, +\infty\}$, also known as the indicator function $1_R(x, y)$, or in bracket notation $\{|(x, y) \in R|\}$ $\square$

Having established the general framework, we now restrict our attention to the convex setting, which is the natural home for optimization. The key insight is that convexity is preserved under infimal composition, so the convex bifunctions form a subcategory of $\mathbf{BiFunc}$ that is closed under the categorical operations.

Definition 2.14 (Convex Bifunction Category $\mathbf{CxBiFn}$). *Let*

$$Q := (\overline{\mathbb{R}}, \leq_Q, \otimes, e)$$

be the extended-Lawvere quantale where:

- $\overline{\mathbb{R}} = \mathbb{R} \cup \{-\infty, +\infty\}$;
- $a \leq_Q b \iff a \geq b$ in the usual order;
- $a \otimes b := a + b$;
- $e := 0$.

The category $\mathbf{CxBiFn}$ is defined as follows:

- **Objects:** *finite-dimensional real vector spaces.*
- **Morphisms:** *A morphism $F : A \rightarrow B$ is a function*

$$F : A \times B \rightarrow \overline{\mathbb{R}}$$

satisfying:

- *F is proper (not identically $+\infty$);*
- *F is convex as a function on the product space $A \times B$;*
- *F is lower semicontinuous.*
- **Composition:** *Given $F : A \rightarrow B$ and $G : B \rightarrow C$, their composite*

$$G \circ F : A \rightarrow C$$

is defined by infimal convolution:

$$(G \circ F)(a, c) := \inf_{b \in B} (F(a, b) + G(b, c)).$$

- **Identity:** *For each object A , the identity morphism*

$$1_A : A \rightarrow A$$

is given by

$$1_A(a, a') = \begin{cases} 0 & \text{if } a = a', \\ +\infty & \text{otherwise.} \end{cases}$$

- *Monoidal product*

- *On objects:*

$$A \otimes B := A \times B.$$

- *On morphisms:* For $F : A \rightarrow C$ and $G : B \rightarrow D$,

$$(F \otimes G)((a, b), (c, d)) := F(a, c) + G(b, d).$$

- *The monoidal unit is the zero-dimensional vector space $\{0\}$.*

Remark 2.4. One may define the analogous category **CvBiFn** which is the category of concave bifunctions. For doing so, one should only change the composition from $\inf$ to $\sup$, this is:

$$(G \circ F)(a, c) := \sup_{b \in B} (F(a, b) + G(b, c)).$$

Every result for **CxBiFn** will hold, in an equivalent way, to the category **CvBiFn**

Remark 2.5. Every finite-dimensional real vector space V is linearly isomorphic to $\mathbb{R}^n$, where $n = \dim V$.

Hence, one may restrict the objects of **CxBiFn** to the family

$$\text{Ob}(\text{CxBiFn}) = \{\mathbb{R}^n \mid n \in \mathbb{N}\}.$$

With this choice, **CxBiFn** becomes a skeletal category, since $\mathbb{R}^n \cong \mathbb{R}^m$ implies $n = m$.

Also, if one considers the isomorphism $n \leftrightarrow \mathbb{R}^n$, **CxBiFn** becomes a prop, this can be made strict by defining the monoidal product on objects as sum.

$$m \otimes n = m + n$$

which is isomorphic to

$$\mathbb{R}^m \otimes \mathbb{R}^m = \mathbb{R}^m \times \mathbb{R}^m = \mathbb{R}^{m+n}$$

Observe that we have constructed a category whose morphisms are interpreted as convex optimization problems. Since every bifunction $F : \mathbb{R}^m \times \mathbb{R}^n \rightarrow \overline{\mathbb{R}}$ can be written as

$$F(x, y) = \inf_z z \quad \text{s.t. } z = F(x, y)$$

This observation is central to the thesis. It shows that there's some sort of duality between relations and optimization. For a single morphism, however, this remains imprecise. The compositional structure yields more substantive insights.

$$(S \circ R)(x, z) = \inf_y \{R(x, y) + S(y, z)\}$$

Which shows that relational composition is precisely variable elimination: composing two relations automatically marginalises over the intermediate variable y by minimising the combined cost. In the binary case this reduces to the existential question of whether a mediating element exists; in the weighted case it asks for the **optimal** such element. Optimization is therefore not imposed on the categorical structure from outside — it emerges from composition itself.

Convex theory also has a very nice duality theory which is of particular interest in algebraic constructs such as this one. We, however, do not include such results in this thesis, leaving it for the interested reader [SS24; Wil15]

The results of this section establish **CxBiFn** as a well-behaved categorical setting for convex optimization. The special classes of morphisms of polyhedral bifunctions inherit this structure while admitting a finite, explicit representation. It is precisely this finiteness that makes a graphical algebra possible: morphisms can be encoded as diagrams composed from a finite set of generators, and the equational theory of the diagrams corresponds exactly to the equivalences between optimization problems. The following section make this precise.

3 POLYHEDRAL RELATIONS

We now turn to a combinatorially richer class of objects: polyhedra and polyhedral cones. These are the fundamental geometric structures underlying linear programming, and they admit a categorical treatment entirely analogous to the one developed for linear and affine relations. The key difference is that composition here is governed by Fourier-Motzkin Elimination (FME): projecting a polyhedron onto a subspace is precisely variable elimination, and it is this operation that categorical composition will simulate diagrammatically.

The theory developed here remains within the category of binary relations, since morphisms are still indicator functions of subsets of $\mathbb{R}^n \times \mathbb{R}^m$, encoding feasibility constraints rather than objective functions. In this sense, **GPA** does not yet enter the weighted relational world, and so is not yet able to express more meaningful optimization problems. This is nonetheless a critical step in the development. Axiomatizing linear inequality constraints diagrammatically, **GPA** provides the Symmetric Monoidal Inequality Theory (SMIT) [BDS21] that will serve as the structural foundation for our final contribution, **GLP**, where these constraints are combined with a linear objective to form full parametric linear programs.

In what follows we describe and show the complete theory of polyhedral relations and polyhedral cones. Just as **AffRel** captured affine space as morphisms, the categories **PCRel** and **PolyRel** capture polyhedral cones and polyhedra as morphisms, with relational composition playing the role of variable elimination.

3.1 SEMANTICS: THE PROP PCREL OF POLYHEDRAL CONES

We begin by recalling the standard definitions of polyhedral cones and the corresponding semantic category.

Definition 3.1. A set $C \subseteq \mathbb{R}^n$ is called a polyhedral cone if there exists a matrix $A \in \mathbb{R}^{m \times n}$ such that

$$C = \{x \in \mathbb{R}^n \mid Ax \geq 0\}.$$

Definition 3.2 (The prop **PCRel** of polyhedral cones). Let k be an ordered field, in this case the real numbers. The prop **PCRel** _{k} is defined as follows:

- Objects are natural numbers $n \in \mathbb{N}$, interpreted as the space k^n .
- A morphism $n \rightarrow m$ is a polyhedral cone $R \subseteq k^n \times k^m$ for which there exist $p \in \mathbb{N}$ and $A \in k^{p \times (n+m)}$ such that

$$R = \left\{ (x, y) \in k^n \times k^m \mid A \begin{pmatrix} x \\ y \end{pmatrix} \geq 0 \right\}.$$

- *Composition is relational composition.*
- *The monoidal product on objects is addition $n \otimes m := n + m$, and on morphisms is the Cartesian product.*

Observe that in comparison with the previous categories of **AffRel**, this category is way more general although it does not completely encode the notion of Affine Spaces. However, it does introduce to us a somewhat unorthodox notion of conical polyhedral relations. That is, the fact that two elements x, y may be related to one another by a relation R such that the set of all pairs xRy form a polyhedral cone. Although this description remains very set-theoretically, it starts to flirt with the idea of x and y being apparted from a specific class of maps. More precisely, instead of thinking of the whole geometry of polyhedral cones, one may think of the pairs (x, y) such that $Ax \geq By$ for some A and B fully defined by the relation R .

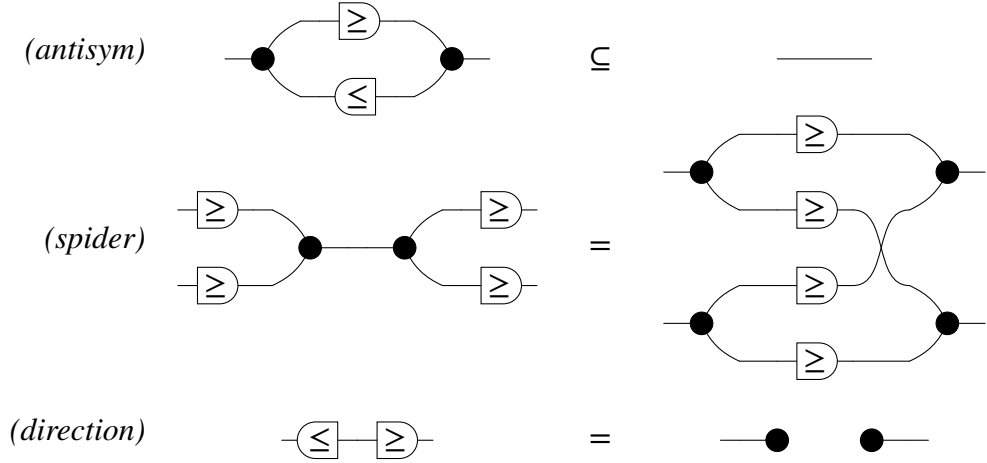
3.2 SYNTAX: THE ALGEBRA $\mathbf{GPA}_{\perp}$

Definition 3.3 (Graphical Polyhedral Algebra). *We call **GPA** the SMT generated by extending **GAA** with the following generator and its semantics:*

$$\llbracket \text{---} \boxed{\geq} \text{---} \rrbracket = \{(x, y) | x, y \in k, x \geq y\}$$

along with the following additional equations:

$$\begin{array}{lll}
 (P0) & \text{---} \boxed{\geq} \text{---} & = \text{---} \boxed{\geq} \text{---} \bigcirc \\
 (P1) & \text{---} \boxed{\geq} \text{---} \bullet & \subseteq \text{---} \bullet \begin{array}{l} \boxed{\geq} \\ \boxed{\geq} \end{array} \\
 (P2) & \bigcirc \text{---} \boxed{\geq} \text{---} & = \begin{array}{l} \boxed{\geq} \\ \boxed{\geq} \end{array} \text{---} \bigcirc \\
 (P3) & \text{---} \boxed{\geq} \text{---} \bullet & = \text{---} \bullet \\
 (P4) & \bigcirc \text{---} & \subseteq \bigcirc \text{---} \boxed{\geq} \text{---} \\
 (P5) \ k > 0 & \text{---} \boxed{k} \text{---} \boxed{\geq} \text{---} & = \text{---} \boxed{\geq} \text{---} \boxed{k} \text{---} \\
 (P6) \ k < 0 & \text{---} \boxed{k} \text{---} \boxed{\geq} \text{---} & = \text{---} \boxed{\leq} \text{---} \boxed{k} \text{---}
 \end{array}$$



Definition 3.4 (Graphical Polyhedral Cones Algebra). *We call $\mathbf{GPA}_{\vdash}$ the SMT generated by GPA but excluding the affine generator $\vdash$ and its associated equations. This fragment axiomatises polyhedral cones, where no translation is present.*

Definition 3.5 (The Free Categories of Polyhedral Diagrams). *We call $\mathbf{FreeP}_{\mathbf{GPA}}$ the PROP freely generated by GPA, and $\mathbf{FreeP}_{\mathbf{GPA}_{\vdash}}$ the PROP freely generated by $\mathbf{GPA}_{\vdash}$.*

3.3 THE ORDER STRUCTURE OF THE INEQUALITY GENERATOR

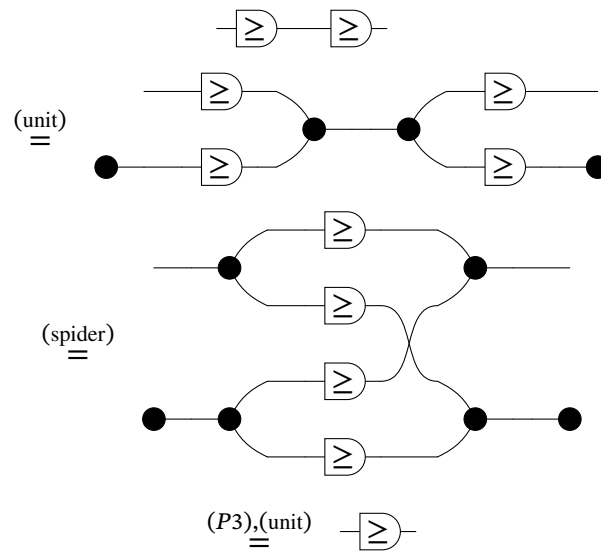
The central computational primitive for reasoning inside this algebra is Fourier–Motzkin Elimination, which projects a polyhedral cone onto a lower-dimensional space by eliminating one variable at a time.

It is worth being explicit about what that algorithm actually does to a system $Ax \geq 0$, because it fixes exactly which diagrammatic rules we have to establish first. To eliminate a variable t , the algorithm divides each row containing t by its coefficient, so that the row becomes a bound on t ; it records whether the bound is an upper or a lower one, according to the sign of that coefficient; and it then replaces the whole collection of bounds by one inequality for each upper–lower pair. Three moves, therefore: rescaling a row, moving a term across a sum, and merging bounds pairwise. Two of them are axioms of Definition 3.3; this section supplies the third and the order laws the axioms omit, and Section 3.4 assembles them into the elimination itself.

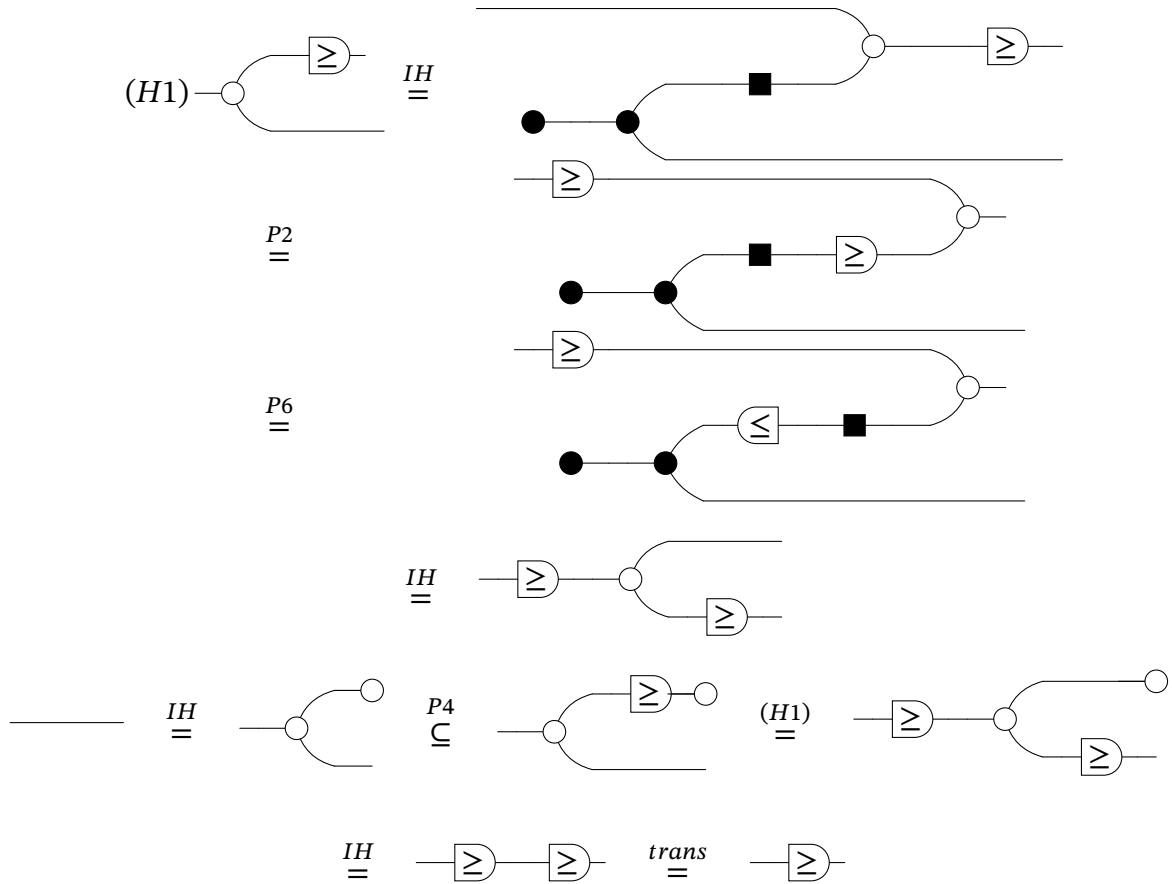
Rescaling and sign are given outright by (P5) and (P6), and the pairwise merge by (spider). What the axioms do not state, and what we need first, is that $\boxed{\geq}$ deserves its name: reflexivity and transitivity are not among them, and antisymmetry is postulated only as the inclusion $\subseteq$.

Lemma 3.1 (The generator is an order). *In $\mathbf{GPA}_{\vdash}$ the generator $\boxed{\geq}$ is reflexive, transitive and antisymmetric.*

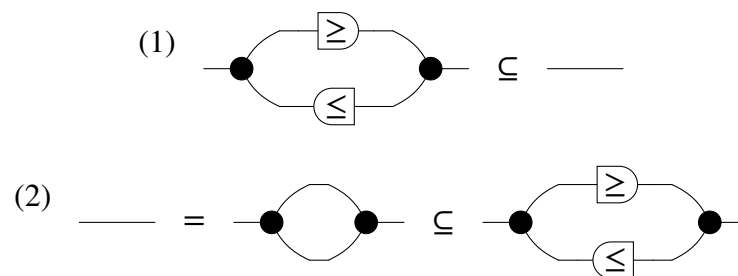
Proof. Transitivity is (spider) with one input fed by the black unit and one output discarded, the plugs being absorbed by (P3) and the black (co)unit laws:



For reflexivity we need a previous results that we call *H1*



Antisymmetry, the converse of the inclusion (antisym):



$$(1) \wedge (2) \implies \text{---} = \text{---} \begin{array}{c} \boxed{\geq} \\ \boxed{\leq} \end{array} \text{---}$$

□

Since $\boxed{\leq}$ is $\boxed{\geq}$ with an antipode on each side, by (P6) at $k = -1$, every axiom comes in a mirrored version, and we use both without further comment.

The move the axioms leave implicit is the transport of a term across a sum: it is the step that turns a row of a system into a bound on a single variable, and it is the content of the next lemma.

Lemma 3.2 (Cap form). *In $\text{GPA}_{\vdash}$,*

$$(C1) \quad \begin{array}{c} \boxed{\geq} \\ \text{---} \end{array} \text{---} \bigcirc \text{---} = \text{---} \bigcirc \text{---} \boxed{\geq} \text{---} = \begin{array}{c} \text{---} \\ \boxed{\geq} \end{array} \text{---} \bigcirc \text{---}$$

$$(C2) \quad \text{---} \bigcirc \text{---} \boxed{\geq} \text{---} \bigcirc = \begin{array}{c} \boxed{\geq} \\ \text{---} \end{array} \text{---} \blacksquare \text{---}$$

The first chain reads $x + u \geq v$ iff $v - u \leq x$: an inequality may be applied before or after a sum indifferently. The second reads $s + u \geq 0$ iff $s \geq -u$, and it is the form we shall use: the diagram $(\boxed{\geq} \oplus \text{id}); \varepsilon$, whose denotation is $\{(\ell, r) \mid \ell \geq r\}$, we call the *cap form* of $\ell \geq r$, so that a row $s + u \geq 0$ of a system is the cap form of the bound $s \geq -u$.

Proof. Bending the second input of the white sum to an output is a bijection on hom-sets, so the first equation may be compared in bent form; the third follows by commutativity of the sum. Closing the bent leg with a white cozero then gives the second equation.

$$(C1) \quad \begin{array}{c} \boxed{\geq} \\ \text{---} \end{array} \text{---} \bigcirc \text{---} = \begin{array}{c} \boxed{\geq} \quad \boxed{\geq} \\ \text{---} \end{array} \text{---} \bigcirc \text{---} = \begin{array}{c} \boxed{\geq} \\ \text{---} \end{array} \text{---} \bigcirc \text{---} \boxed{\geq} \text{---}$$

$$= \text{---} \bigcirc \text{---} \boxed{\geq} \text{---} \boxed{\geq} \text{---} = \text{---} \bigcirc \text{---} \boxed{\geq} \text{---}$$

$$(C2) \quad \text{---} \bigcirc \text{---} \boxed{\geq} \text{---} \bigcirc \stackrel{(C1)}{=} \begin{array}{c} \boxed{\geq} \\ \text{---} \end{array} \text{---} \bigcirc \text{---} \bigcirc \stackrel{\text{(white cap)}}{=} \begin{array}{c} \boxed{\geq} \\ \text{---} \end{array} \text{---} \blacksquare \text{---} \bigcirc \text{---} \bigcirc$$

□

The same bending turns a two-sided system $Ax \geq By$ into a homogeneous one. This is the presentation in which elimination takes place, since it has a single block of rows and no distinguished side.

Lemma 3.3 (Cone form). *Let $A : n \rightarrow p$ and $B : m \rightarrow p$ be matrices. Bending the m outputs of A ; $\boxed{\geq}$; B^{op} into inputs yields the cone $[A \ -B]; \boxed{\geq}^{\oplus p}; \text{coz}^{\oplus p}$, and unbending recovers the original diagram.*

That is, $\exists v. Ax \geq v, v = By$ iff $Ax - By \geq 0$. Rescaling the rows of a cone by positive constants leaves it unchanged, since $Dz \geq 0$ iff $z \geq 0$ for diagonal $D > 0$; this is (P5) again, applied row by row.

Proof. Bending the comatrix exposes a white sum with an antipode on one leg, and the cap form pushes the inequality past it.

□

The third move, the pairwise merge, is (spider). It is the only one of the three that is not local: on its left-hand side a black node forces a single intermediate value below every x_i and above every y_j , and on its right-hand side that value is gone and only the relations $x_i \geq y_j$ survive. The axiom is therefore the elimination of an intermediate variable in the form discussed in the introduction, and the number of rows grows from $a + b$ to ab exactly as it does in the classical algorithm.

3.4 FOURIER–MOTZKIN ELIMINATION

With the three moves available we can eliminate a single variable. Let $M \in \mathbb{R}^{p \times q}$ have first column a , and split the row indices into $P = \{i \mid a_i > 0\}$, $N = \{j \mid a_j < 0\}$ and $Z = \{i \mid a_i = 0\}$.

Proposition 3.1 (Fourier-Motzkin Elimination). *The following diagrammatic procedure is equivalent to perform the FME algorithm*

Proof. First consider one middle wire; ellipses denote parallel repetitions.

For positive coefficients,

$$\begin{aligned}
 \boxed{\geq} - \boxed{d} &\stackrel{\text{inversion lemma}}{=} \boxed{\geq} - \boxed{d^{-1}} \stackrel{(P5)}{=} \boxed{d^{-1}} - \boxed{\geq} \stackrel{\text{inversion lemma}}{=} \boxed{d} - \boxed{\geq} \\
 &\stackrel{(P5)}{=} \boxed{d} - \boxed{\geq} = \boxed{\geq} - \boxed{d}
 \end{aligned}$$

and for negative coefficients,

$$\begin{aligned}
 \boxed{\geq} - \boxed{d} &\stackrel{\text{inversion lemma}}{=} \boxed{\geq} - \boxed{d^{-1}} \stackrel{(P6)}{=} \boxed{d^{-1}} - \boxed{\leq} \stackrel{\text{inversion lemma}}{=} \boxed{d} - \boxed{\leq} \\
 &\stackrel{(P6)}{=} \boxed{d} - \boxed{\geq} = \boxed{\leq} - \boxed{d}
 \end{aligned}$$

Zero legs detach:

$$\begin{aligned}
 \boxed{\geq} - \boxed{0} &\stackrel{(\text{zero sc.}), (\text{slide})}{=} \boxed{\geq} - \bigcirc - \bullet \\
 \boxed{0} - \boxed{\geq} &\stackrel{(\text{zero sc.})}{=} \bullet - \bigcirc - \boxed{\geq} \\
 \text{---} \bullet - \bullet &\stackrel{(\text{unit})}{=} \text{---} \quad \bullet - \bullet \text{---} \stackrel{(\text{unit})}{=} \text{---}
 \end{aligned}$$

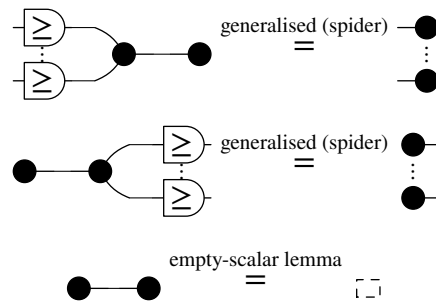
Bend the reversed legs:

$$\boxed{\leq} \text{---} \stackrel{(\text{slide})}{=} \boxed{\geq} \text{---}$$

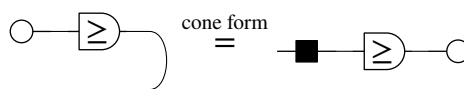
The nonzero part now rewrites as follows.

$$\begin{aligned}
 &\begin{array}{c} \boxed{d_1} - \boxed{\geq} \\ \vdots \\ \boxed{d_p} - \boxed{\geq} \end{array} \text{---} \bullet \text{---} \bullet \begin{array}{c} \boxed{\geq} - \boxed{e_1} \\ \vdots \\ \boxed{\geq} - \boxed{e_q} \end{array} \stackrel{\text{generalised (spider)}}{=} \begin{array}{c} \boxed{d_1} - \bullet \\ \vdots \\ \boxed{d_p} - \bullet \end{array} \begin{array}{c} \boxed{\geq} \\ \vdots \\ \boxed{\geq} \end{array} \begin{array}{c} \bullet \\ \vdots \\ \bullet \end{array} \begin{array}{c} \boxed{\geq} - \boxed{e_1} \\ \vdots \\ \boxed{\geq} - \boxed{e_q} \end{array} \\
 &\stackrel{\text{inversion lemma}}{=} \begin{array}{c} \boxed{d_1^{-1}} - \bullet \\ \vdots \\ \boxed{d_p^{-1}} - \bullet \end{array} \begin{array}{c} \boxed{\geq} \\ \vdots \\ \boxed{\geq} \end{array} \begin{array}{c} \bullet \\ \vdots \\ \bullet \end{array} \begin{array}{c} \boxed{e_1^{-1}} \\ \vdots \\ \boxed{e_q^{-1}} \end{array} \\
 &\stackrel{(\text{matrix def.})}{=} \boxed{H} - \boxed{\geq} - \boxed{K}
 \end{aligned}$$

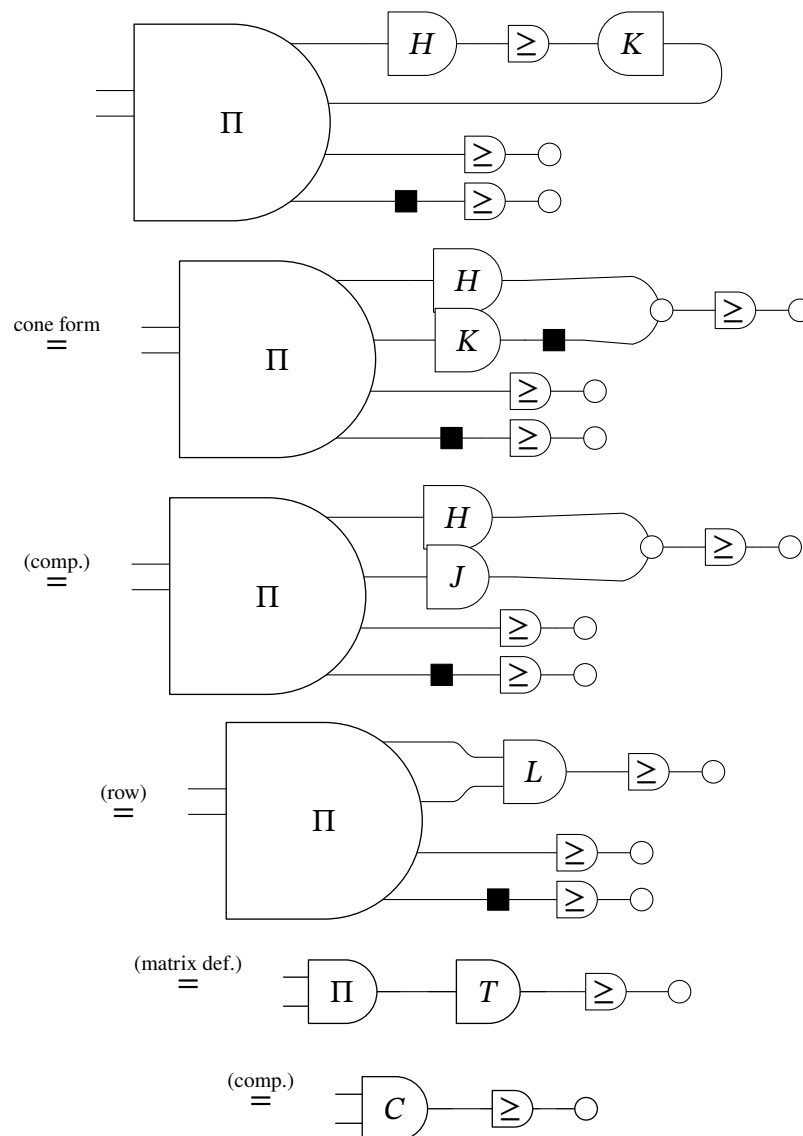
For an empty family of bounds,



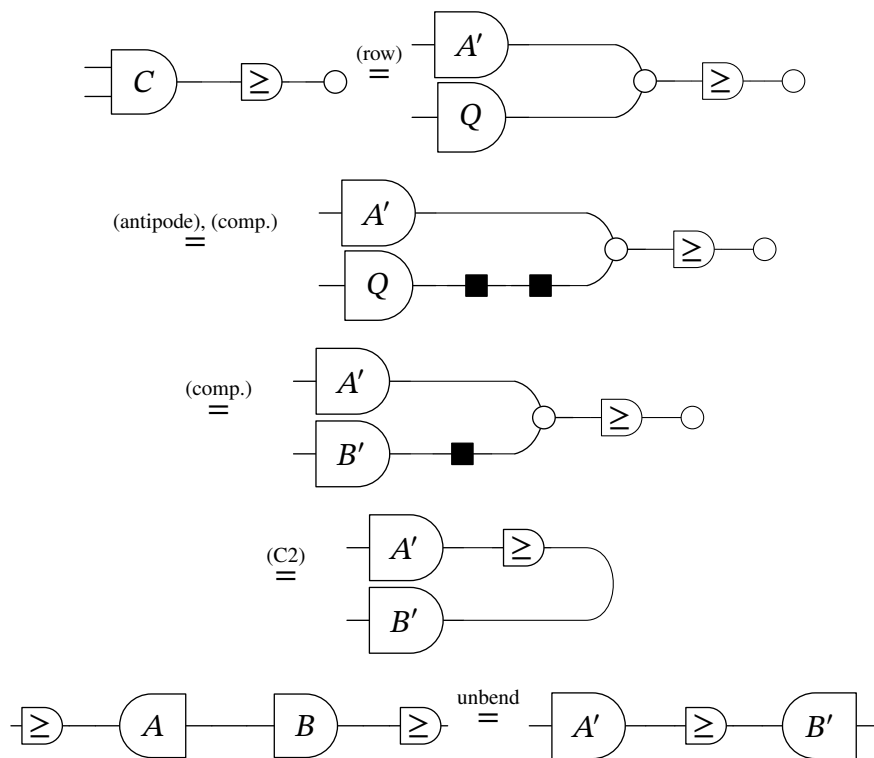
Bend the detached output constraints as well:



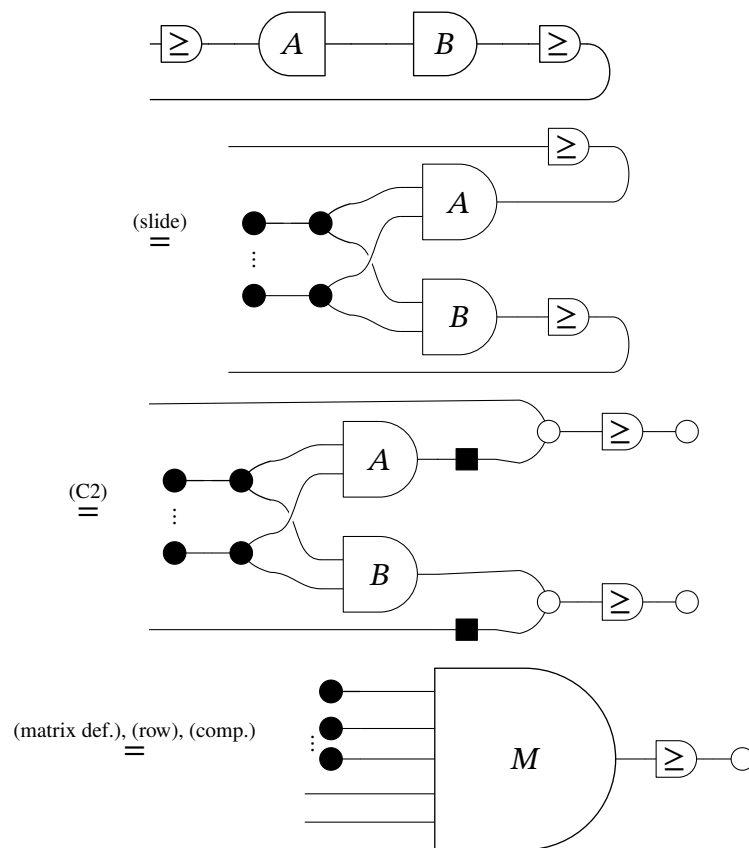
Reattach the zero branches; the permutation restores their boundary positions.



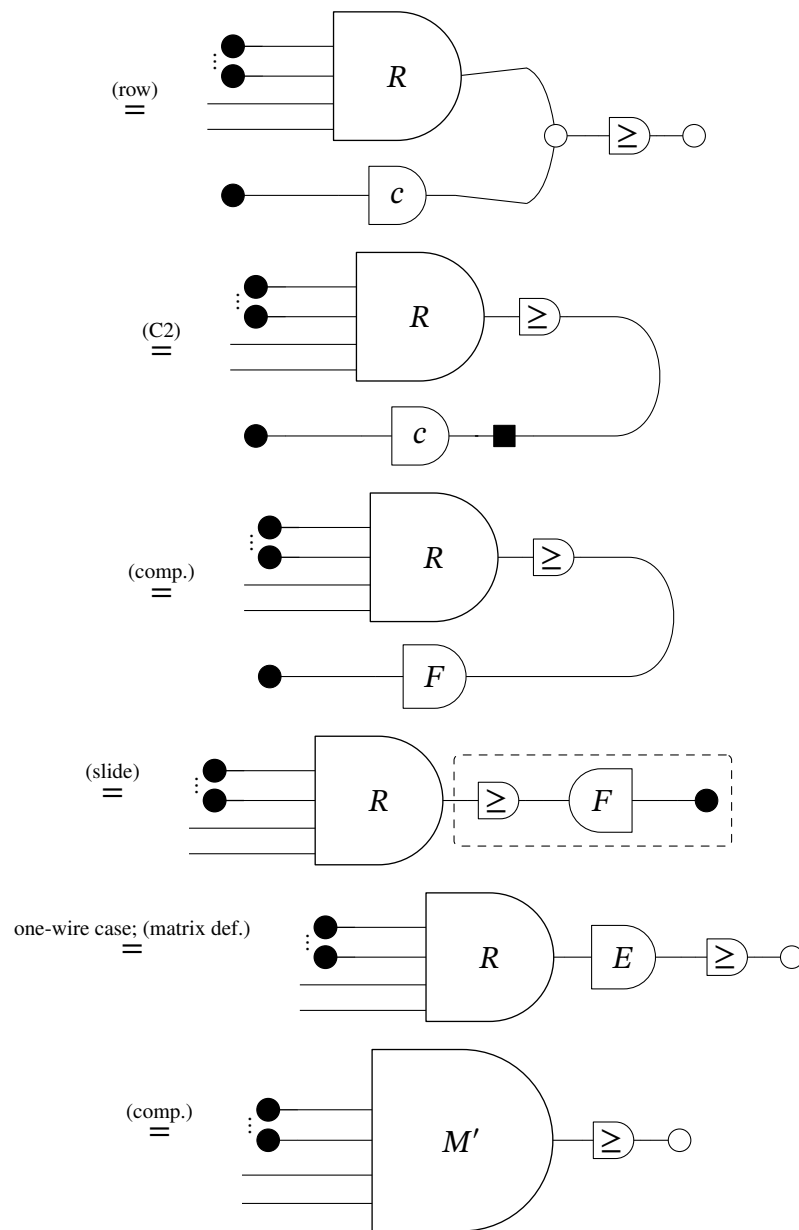
Restore the original input-output boundary:



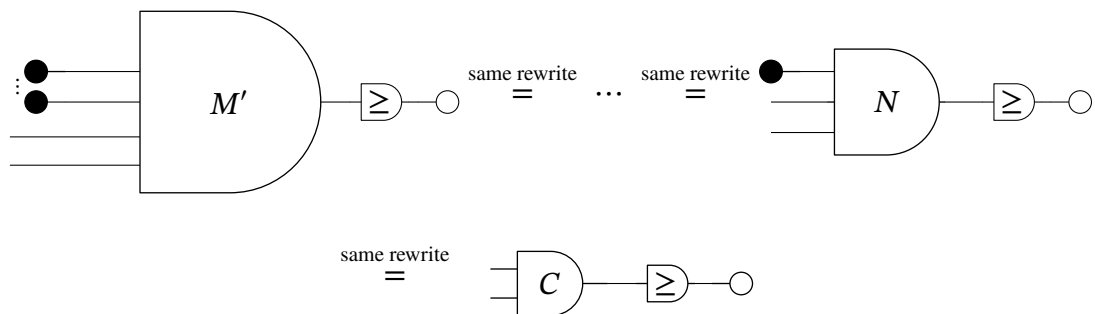
Now bend an arbitrary middle bundle and expose its independent sources.



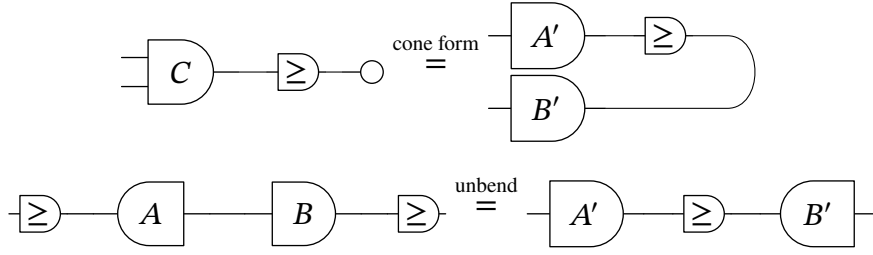
Separate one source.



Repeat the same local rewrite until no source remains.



Apply the boundary-restoration chain above:

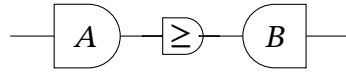


□

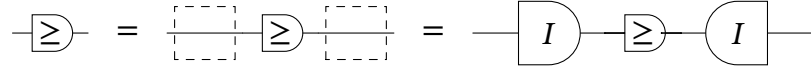
A pair (x, y) is related by $\overline{\geq}; g; \overline{\geq}$ precisely when there are v, w with $x \geq v$, $(v, w) \in \llbracket g \rrbracket$ and $w \geq y$, so the two bound variables v, w are what the elimination has to remove.

With FME available as a diagrammatic operation, we can now prove the normal form theorem for $\mathbf{GPA}_{\perp}$. We proceed by structural induction, handling generators, composition, and tensor product in turn, using FME steps to push all inequality generators into a canonical block form.

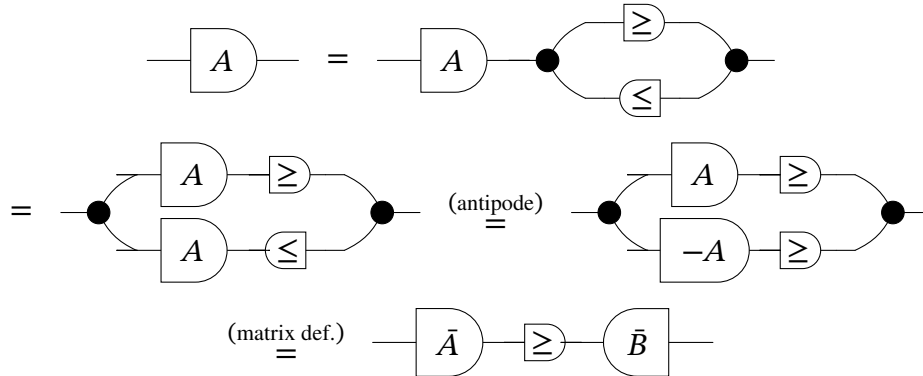
Theorem 3.1 (First Normal Form). *Every diagram in $\mathbf{GPA}_{\perp}$ admits an isomorphic representation, modulo the axioms, of the following form:*



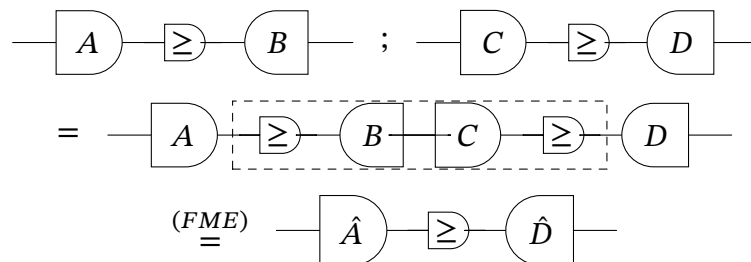
Proof. We proceed by structural induction. The two base cases are as follows. First, the $\geq$ generator:



Second, for any diagram c in $\mathbf{GLA}$:



For the inductive step, we consider composition of two diagrams $c; d$ in $\mathbf{GPA}_{\perp}$. By the inductive hypothesis, both c and d are already in normal form:



And for the tensor product:

$$\begin{array}{c} \text{---} \square A \text{---} \bigcirc \geq \text{---} \square B \text{---} \\ \text{---} \square C \text{---} \bigcirc \geq \text{---} \square D \text{---} \end{array} = \text{---} \square \bar{A} \text{---} \bigcirc \geq \text{---} \square \bar{B} \text{---}$$

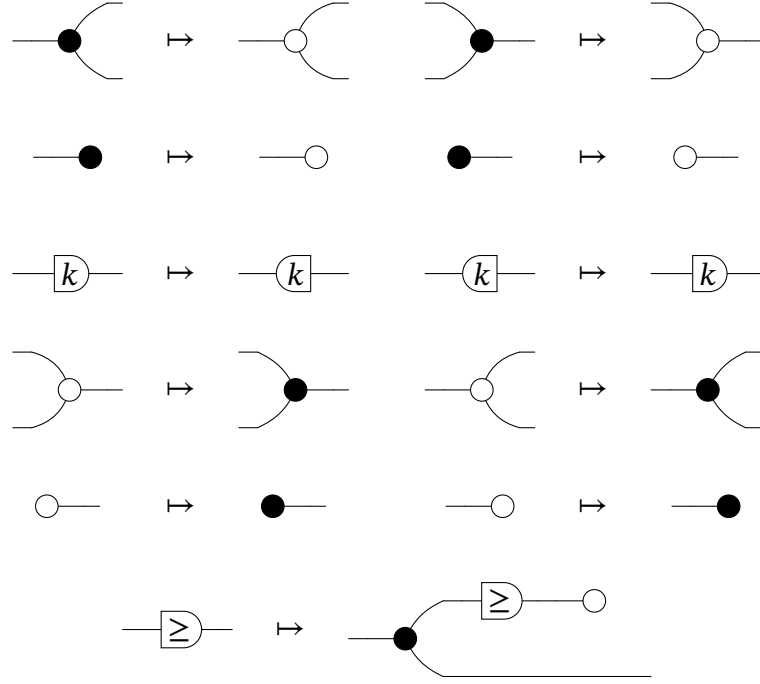
□

Working with a normal form defined up to isomorphism is in principle weaker than requiring syntactic equality of terms. This choice is motivated by the compact-closed structure, which provides a canonical bijection $\text{Hom}(A, B) \cong \text{Hom}(\mathbb{1}, A^* \otimes B)$, allowing input wires to be treated symmetrically with output wires but only up to this isomorphism, not up to equality. As shown in [BDS21], the same completeness argument can be carried out using strict diagrammatic equality; however, the derivations become significantly more involved. Since every result established for the isomorphic normal form carries over to the strict one, the only difference being the need to bend wires at each step, we adopt the more convenient convention throughout.

This idea was already encountered in the treatment of Graphical Quadratic Algebra [Ste+24], but it is worth pausing to appreciate its significance. The compact-closed structure does not merely allow a more convenient notation; it enables diagrammatic reasoning to serve as a rigorous replacement for coordinate-based algebraic manipulation while remaining extremely intuitive: bending a wire to the left or to the right does exactly what one would expect. Even more, a statement that would classically require several lines of matrix algebra, tracking indices, transposing, and substituting, can instead be verified by a sequence of local wire manipulations, each justified by a single axiom.

This first normal form expresses every diagram as a system of inequalities applied to a linear map. A dual normal form can be obtained by applying the polar operator, which we now define. The polar operator plays the role of duality in the polyhedral setting, exchanging the H -representation (intersection of halfspaces) with the V -representation (conic hull of generators), in direct analogy with the Farkas lemma [BDZ21].

Definition 3.6 (Polar Operator). *The polar operator $-^\circ$ is the involution on generators defined by:*



Its action on diagrams is defined functorially by $(a \circ b)^\circ = a^\circ \circ b^\circ$.

Proposition 3.2 (Second Normal Form). *Every diagram $s \in \mathbf{GPA}_\perp$ can be written as:*

$$\left(\text{---} \boxed{A} \text{---} \boxed{\geq} \text{---} \boxed{B} \text{---} \right)^\circ = \text{---} \boxed{A^T} \text{---} \bullet \begin{cases} \boxed{\geq} \text{---} \circ \\ \boxed{B^T} \text{---} \end{cases}$$

Proof. Apply the polar operator to the first normal form of every diagram $c \in \mathbf{GPA}_\perp$. Since the polar operator is an involution and acts functorially, the result is again a valid diagram in $\mathbf{GPA}_\perp$ in the claimed form. $\square$

The second normal form expresses every diagram as a conic combination of column vectors, which is precisely the V -representation of a polyhedral cone. This is the representation that will drive the completeness proof: two diagrams that are semantically equal must have the same conic hull, and the axioms of $\mathbf{GPA}_\perp$ are exactly sufficient to witness this equality diagrammatically.

3.5 PRESENTATION RESULTS FOR PCREL

We now develop the presentation theory for the homogeneous case, proceeding to establish the three properties that together yield the presentation of $\mathbf{PCRel}$.

Lemma 3.4 (Soundness). *If $s = t$ in $\text{FreeP}_{\mathbf{GPA}_\perp}$, then $[s] = [t]$ in $\mathbf{PCRel}$.*

Proof. All axioms except those involving the newly introduced $\geq$ generator are already present in $\mathbf{GAA}$ and therefore verified. It suffices to check that the new axioms preserve the semantics:

(P1)

$$\begin{aligned}
\left[\text{---} \boxed{\geq} \text{---} \bullet \right] &= \left\{ (x, \binom{y_1}{y_2}) \mid x \geq y_1 \wedge x \geq y_2 \wedge y_1 = y_2 \right\} \\
&\subseteq \left\{ (x, \binom{y_1}{y_2}) \mid x \geq y_1 \wedge x \geq y_2 \right\} = \left[\text{---} \bullet \begin{array}{l} \boxed{\geq} \\ \boxed{\geq} \end{array} \right]
\end{aligned}$$

(P2)

$$\begin{aligned}
\left[\begin{array}{l} \text{---} \end{array} \bigcirc \text{---} \boxed{\geq} \text{---} \right] &= \left\{ \left(\binom{x_1}{x_2}, y \right) \mid x_1 + x_2 \geq y \right\} = \left\{ \left(\binom{x_1}{x_2}, y \right) \mid \exists c, d. x_1 + x_2 \geq c + d \wedge c + d = y \right\} \\
&= \left\{ \left(\binom{x_1}{x_2}, y \right) \mid \exists \hat{c}, \hat{d}. x_1 \geq \hat{c} \wedge x_2 \geq \hat{d} \wedge \hat{c} + \hat{d} = y \right\} = \left[\begin{array}{l} \boxed{\geq} \\ \boxed{\geq} \end{array} \bigcirc \text{---} \right]
\end{aligned}$$

(P3)

$$\left[\text{---} \boxed{\geq} \text{---} \bullet \right] = \{ (x, \bullet) \mid \exists c. x \geq c \} \stackrel{\text{tautology}}{=} \{ (x, \bullet) \mid x \in \mathbb{R} \} = \left[\text{---} \bullet \right]$$

(P4)

$$\left[\bigcirc \text{---} \right] = \{ (x, \bullet) \mid x = 0 \} \subseteq \{ (x, \bullet) \mid x \geq 0 \} = \left[\bigcirc \text{---} \boxed{\geq} \text{---} \right]$$

(P5), (P6)

For $k > 0$:

$$\begin{aligned}
\left[\text{---} \boxed{k} \text{---} \boxed{\geq} \text{---} \right] &= \{ (x, y) \mid kx \geq y \wedge k > 0 \} = \left\{ (x, y) \mid x \geq \frac{y}{k} \wedge k > 0 \right\} \\
&= \{ (x, y) \mid \exists c. x \geq c \wedge kc = y \wedge k > 0 \} = \left[\text{---} \boxed{\geq} \text{---} \boxed{k} \text{---} \right]
\end{aligned}$$

For $k < 0$: the same computation, dividing by the negative k flips the inequality, so

$$\left[\text{---} \boxed{k} \text{---} \boxed{\geq} \text{---} \right] = \left[\text{---} \boxed{\leq} \text{---} \boxed{k} \text{---} \right]$$

(ANTISYM)

$$\begin{aligned}
 \left[\text{---} \bullet \begin{array}{c} \text{---} \boxed{\geq} \text{---} \\ \text{---} \boxed{\leq} \text{---} \end{array} \bullet \text{---} \right] &= \{(x, y) \mid x \geq y \wedge x \leq y\} \\
 &= \{(x, y) \mid x = y, x, y \in \mathbb{R}\} \subseteq \{(x, y) \mid x = y, x, y \in \mathbb{R}\} = \left[\text{---} \right]
 \end{aligned}$$

(SPIDER)

$$\begin{aligned}
 \left[\begin{array}{c} \text{---} \boxed{\geq} \text{---} \\ \text{---} \boxed{\geq} \text{---} \end{array} \bullet \text{---} \bullet \begin{array}{c} \text{---} \boxed{\geq} \text{---} \\ \text{---} \boxed{\geq} \text{---} \end{array} \right] &= \left\{ \left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} \right) \mid \exists z. x_1 \geq z \wedge x_2 \geq z \wedge z \geq y_1 \wedge z \geq y_2 \right\} \\
 &= \left\{ \left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} \right) \mid x_1 \geq y_1 \wedge x_1 \geq y_2 \wedge x_2 \geq y_1 \wedge x_2 \geq y_2 \right\} = \left[\begin{array}{c} \text{---} \bullet \begin{array}{c} \text{---} \boxed{\geq} \text{---} \\ \text{---} \boxed{\geq} \text{---} \end{array} \bullet \begin{array}{c} \text{---} \boxed{\geq} \text{---} \\ \text{---} \boxed{\geq} \text{---} \end{array} \bullet \text{---} \\ \text{---} \bullet \begin{array}{c} \text{---} \boxed{\geq} \text{---} \\ \text{---} \boxed{\geq} \text{---} \end{array} \bullet \begin{array}{c} \text{---} \boxed{\geq} \text{---} \\ \text{---} \boxed{\geq} \text{---} \end{array} \bullet \text{---} \end{array} \right]
 \end{aligned}$$

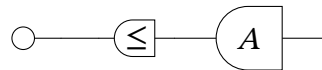
(DIRECTION)

$$\left[\text{---} \boxed{\leq} \text{---} \boxed{\geq} \text{---} \right] = \{(x, y) \mid \exists z. z \geq y \wedge z \geq x\} = \{(x, y) \mid x, y \in \mathbb{R}\} = \left[\text{---} \bullet \quad \bullet \text{---} \right]$$

□

Lemma 3.5 (Expressivity). *For every $f : m \rightarrow n$ in $\mathbf{PCRel}$ there exists a Σ -term $s : m \rightarrow n$ in $\mathbf{Hom}_{\mathbf{FreeP}_{\mathbf{GPA}} \vdash} (m, n)$ such that $[s] = f$.*

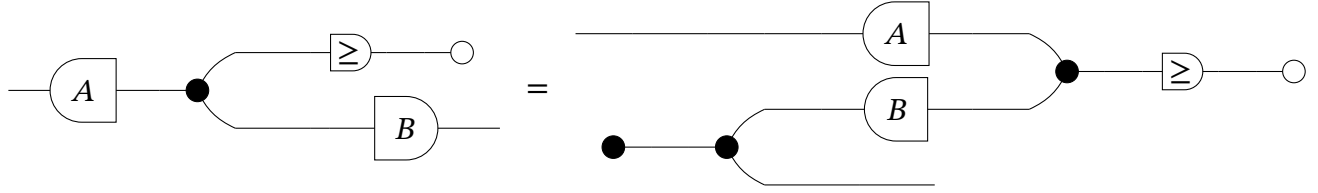
Proof. By Definition 3.1, every polyhedral cone has the form $\{x \mid Ax \geq 0, x \in \mathbb{R}^n\}$. This is precisely the interpretation of the diagram:



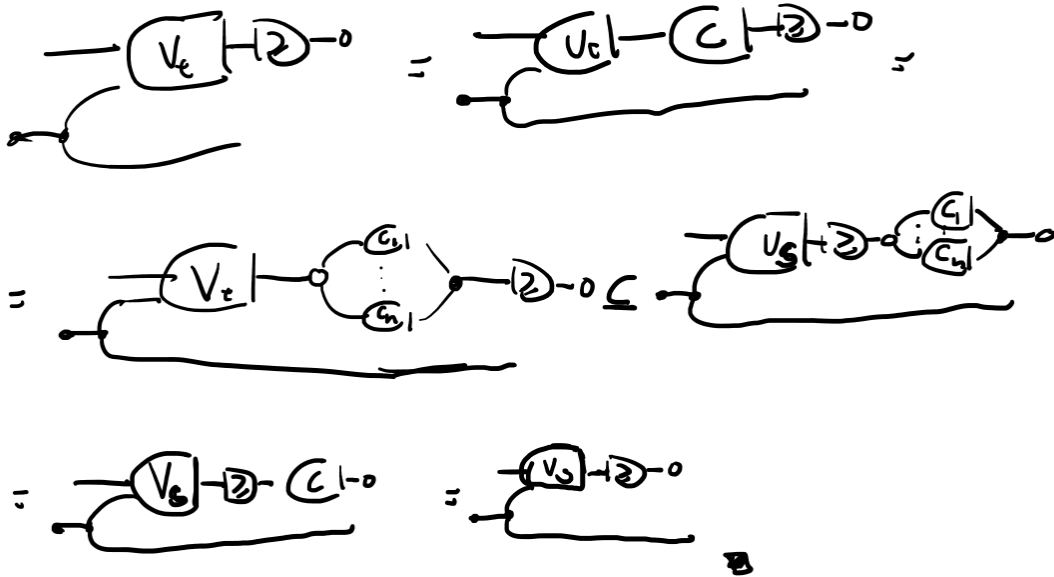
□

Lemma 3.6 (Completeness). *If $[s] = [t]$ in $\mathbf{PCRel}$, then $s = t$ in $\mathbf{FreeP}_{\mathbf{GPA}} \vdash$.*

Proof. By the second normal form, we write s and t in V -representation form:



The semantics of such a diagram is the conic hull of the column vectors of V . Since $[s] = [t]$ implies in particular $[s] \subseteq [t]$, the conic combinations of V_t are a subset of those of V_s . Hence there exists a non-negative matrix C such that $CV_t = V_s$. The diagrammatic witness is:



The inclusion $[s] \subseteq [t]$ follows by the same argument, yielding equality. $\square$

Theorem 3.2. PCRel is presented by $\text{FreeP}_{\text{GPA}_{\perp}}$.

Proof. Soundness, expressivity, and completeness were established by the preceding three lemmas. Together they imply that the canonical PROP morphism is an isomorphism. $\square$

This settles the homogeneous case: every polyhedral cone is diagrammatically expressible, and two diagrams denote the same cone exactly when the axioms can prove them equal. Cones are, however, a narrow class, they always contain the origin, so they cannot represent an arbitrary feasible region or, later, an arbitrary linear program. The next step lifts this result from cones to general polyhedra, replacing the homogeneous inequalities $Ax \geq 0$ with the affine $Ax \geq b$, and the machinery developed above will carry over almost unchanged.

3.6 SEMANTICS: THE PROP POLYREL OF POLYHEDRA

Now we start the presentation of the broader class of polyhedral relations by recalling the definition of a polyhedron.

Definition 3.7. A set $P \subseteq \mathbb{R}^n$ is called a polyhedron if there exist $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ such that

$$P = \{x \in \mathbb{R}^n \mid Ax \geq b\}.$$

Note that a polyhedral cone is a special case of a polyhedron with $b = 0$. The distinction is algebraically significant: cones are closed under non-negative scaling and admit a purely homogeneous representation, while general polyhedra may be translated. This distinction is reflected in the two graphical algebras we now define, where $\mathbf{GPA}_{\vdash}$ handles the homogeneous case and $\mathbf{GPA}$ extends it with an affine generator.

In that light, one can define the category of polyhedral relations as:

Definition 3.8 (The prop $\mathbf{PolyRel}$ of polyhedra). Let k be an ordered field. The prop $\mathbf{PolyRel}_k$ is defined as follows:

- Objects are natural numbers $n \in \mathbb{N}$, interpreted as the space k^n .
- A morphism $n \rightarrow m$ is a polyhedron $R \subseteq k^n \times k^m$ for which there exist $p \in \mathbb{N}$, $A \in k^{p \times (n+m)}$, and $b \in k^p$ such that

$$R = \left\{ (x, y) \in k^n \times k^m \mid A \begin{pmatrix} x \\ y \end{pmatrix} + b \geq 0 \right\}.$$

- Composition is relational composition.
- The monoidal product on objects is addition $n \otimes m := n + m$, and on morphisms is the Cartesian product.

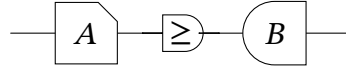
Notice how, the same way that $\mathbf{PCRel}$ offered us a more relational way to think about Polyhedral Cones, so does $\mathbf{PolyRel}$. Indeed, one may think of polyhedra as closed spaces in some geometry that respects some linear constraints. We argue, however, that this definition is overly convoluted, and a more natural way to denote such objects is to think in terms of relations R , such that $xRy \iff Ax + b \geq By + c$ for some matrices A, B and vectors b, c . Once again, we move from a set-theoretic heavy description to a more logical one.

3.7 SYNTAX: THE ALGEBRA $\mathbf{GPA}$

We now extend the theory from polyhedral cones to general polyhedra by reintroducing the affine generator $\vdash$. Geometrically, this corresponds to allowing translations: a polyhedron is

a translated polyhedral cone, and the generator $\vdash$ provides the syntactic means to encode the right-hand side vector b in $Ax \geq b$. The normal form, soundness, and expressivity arguments all extend smoothly from the homogeneous case.

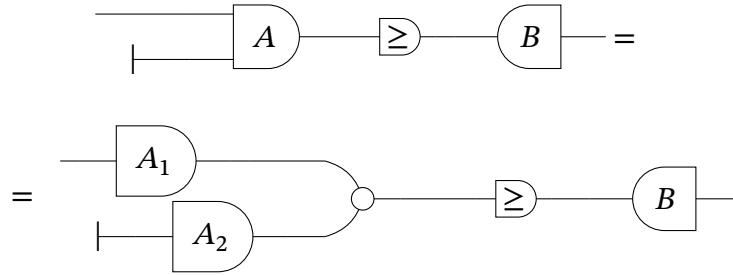
Theorem 3.3 (Normal Form). *For every diagram c in $\mathbf{GPA}$, there exists a diagram d such that $c = d$ modulo the axioms and d is of the form:*



Proof. The derivation proceeds directly. Let c be a diagram of $\mathbf{GPA}$. Then we can collect all $\vdash$ on the left such that c' is a diagram of $\mathbf{GPA}_{\vdash}$



From Theorem 3.1 we know that c' admits a normal form in terms of matrices A and B . Breaking down the matrix A in columns we get:



thus the thesis. □

3.8 PRESENTATION RESULTS FOR POLYREL

Lemma 3.7 (Soundness). *If $s = t$ in $\text{FreeP}_{\mathbf{GPA}}$, then $[s] = [t]$ in $\mathbf{PolyRel}$.*

Proof. This follows immediately from the soundness of $\mathbf{PCRel}$. Since the generator $\vdash$ introduces no new axioms beyond those already verified for $\mathbf{PCRel}$ and $\mathbf{GAA}$, and both of those theories are sound in $\mathbf{PolyRel}$, soundness carries over directly. □

Lemma 3.8 (Expressivity). *For every $f : m \rightarrow n$ in $\mathbf{PolyRel}$ there exists a Σ -term $s : m \rightarrow n$ in $\text{Hom}_{\text{FreeP}_{\mathbf{GPA}}}(m, n)$ such that $[s] = f$.*

Proof. By Definition 3.7, every polyhedron has the form $\{x \mid Ax \geq b\}$. Appealing to this representation directly, the polyhedron is expressed as the interpretation of the following diagram, where the generator $\vdash$ encodes the right-hand side vector b :

☐

Proof. Every polyhedron P can be homogenised by introducing a slack variable, reducing the affine case to the conic one $P^H = \{(x, y) \in k^{n+1} | Ax + by \geq 0, y \geq 0\}$ which admits a normal form (Theorem 3.1). Under this homogenisation, for each two diagrams $s, t \in \mathbf{GPA}$ with $[s] = [t]$ in $\mathbf{PolyRel}$, there are $s', t' \in \mathbf{GPA}_\perp$ such that $[s'] = [s]^H, [t'] = [t]^H$. Since two polyhedrals P, Q are equal iff $P^H = Q^H$, then s and t have equal semantics in $\mathbf{PCRel}$. Completeness of $\mathbf{GPA}_\perp$ then gives equality of the homogenised diagrams, and de-homogenising recovers equality of s and t in $\mathbf{FreeP}_{\mathbf{GPA}}$.

The diagram illustrates the distributive law $A(b + c) = (A + b)c$ in the lambda calculus, showing the equivalence of two expressions through a series of transformations.

Top Expression (Left): Represents $A(b + c)$. It consists of a lambda abstraction $\lambda x. A$ (a box labeled A with an input line) followed by an application node (a circle) that takes the output of $\lambda x. A$ and the input b (a box labeled b with an input line). The result is then passed to a lambda abstraction $\lambda y. B$ (a box labeled B with an input line).

Top Expression (Right): Represents $(A + b)c$. It consists of a lambda abstraction $\lambda x. A$ (a box labeled A with an input line) followed by an application node (a circle) that takes the output of $\lambda x. A$ and the input b (a box labeled b with an input line). The result is then passed to a lambda abstraction $\lambda y. B$ (a box labeled B with an input line).

Transformation 1: The first transformation shows the equivalence between the two expressions. The input c (represented by a line with a dot) is introduced into the expression.

Transformation 2: The second transformation shows the equivalence between the two expressions. The input c is introduced into the expression.

Transformation 3: The third transformation shows the equivalence between the two expressions. The input c is introduced into the expression.

☐

Theorem 3.4. *PolyRel is presented by $\text{FreeP}_{\text{GPA}}$.*

Proof. By the preceding three lemmas. □

This is the result the rest of the thesis leans on: every polyhedral relation, not just the conic ones from earlier, has a diagrammatic representative, and semantic equality between polyhedra can always be certified by a finite rewriting sequence in **GPA**. It is this presentation, together with its normal form, that Chapter 4 builds on when it adds a single new generator for the objective function.

Remark 3.1 (Invariance under non-negative matrices). *Whereas the linearity axiom shows the linear generator to be exactly invariant under column-stochastic reweightings (Remark 4.1), the order generator ge enjoys a related but weaker property with respect to non-negative matrices. Let $A \in \mathbb{R}^{n \times m}$ with $A_{ij} \geq 0$ for all i, j . If $x, y \in \mathbb{R}^m$ satisfy $x \geq y$ componentwise, then $Ax \geq Ay$ componentwise: for each row i ,*

$$(Ax)_i - (Ay)_i = \sum_j A_{ij}(x_j - y_j) \geq 0,$$

since every term is a product of a non-negative entry and a non-negative difference. Diagrammatically, this is the inclusion

$$\text{matrix}[A] ; ge^{\otimes n} \subseteq ge^{\otimes m} ; \text{matrix}[A]^{\otimes 2},$$

*verified directly in the semantics of **PolyRel**, exactly as the axioms of **GPA** are themselves verified. Unlike Remark 4.1, this is only an inclusion and not an equality: a non-negative matrix need not reflect the order, only preserve it, so $Ax \geq Ay$ can hold even when $x \geq y$ fails (take $A = 0$). In other words, a non-negative matrix always goes through the order generator, certifying that any inequality derived in **GPA** remains valid after left multiplication by A .*

4 THE LINEAR PROGRAMMING CATEGORY

The theory developed so far has extended the categorical framework by adding a new class of morphisms. We now introduce the final and most expressive theory of the thesis: a graphical algebra for parametric linear programs. Linear programs combine both the inequality structure of polyhedra and the notion of an objective function, and their value functions are precisely the polyhedral convex functions. The central result of this section is the presentation of a complete monoidal theory for right-hand side parametric linear programming.

4.1 SEMANTICS: THE PROP $\mathbf{LPREL}$

We begin with several classical results that establish the tight relationship between piecewise affine convex functions, polyhedral functions, and parametric linear programs. These will serve as the semantic foundation for the graphical algebra introduced in the following section, and they justify describing the morphisms of this chapter's semantic category, $\mathbf{LPREL}$, interchangeably as parametric linear programs or as polyhedral functions.

Definition 4.1 (Piecewise Affine Convex Function). *A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is called piecewise affine convex if there exist finitely many affine functions $a_i^\top x + b_i$, $i = 1, \dots, k$, such that*

$$f(x) = \max_{i=1, \dots, k} \{a_i^\top x + b_i\}.$$

Definition 4.2 (Polyhedral Function). *A function $f : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$ is polyhedral if its epigraph*

$$\text{epi}(f) := \{(x, t) \mid t \geq f(x)\}$$

is a polyhedron.

Definition 4.3 (RHS Parametric Linear Program). *A right-hand side parametric linear program has value function*

$$v(b) := \inf_x \{c^\top x \mid Ax \geq b\}.$$

The following two propositions establish that all three notions above are equivalent descriptions of the same class of functions. This equivalence is the semantic cornerstone of the section: it is what allows us to treat polyhedral functions and parametric linear programs as morphisms in the same category.

Proposition 4.1. *A function is piecewise affine convex if and only if it is a polyhedral function.*

Proof. ($\Rightarrow$) Let $f(x) = \max_{i=1, \dots, k} (a_i^\top x + b_i)$. Then

$$\text{epi}(f) = \bigcap_{i=1}^k \{(x, t) \mid t \geq a_i^\top x + b_i\},$$

which is an intersection of finitely many halfspaces, hence a polyhedron.

($\Leftarrow$) If f is polyhedral, its epigraph is a polyhedron, so there exist finitely many affine inequalities $t \geq a_i^\top x + b_i$ such that $f(x) = \max_{i=1,\dots,k} (a_i^\top x + b_i)$. Thus f is piecewise affine and convex. $\square$

Proposition 4.2. *A function $v : \mathbb{R}^m \rightarrow \mathbb{R} \cup \{+\infty\}$ is piecewise affine convex if and only if it is the value function of a right-hand side parametric linear program.*

Proof. ($\Rightarrow$) Let $v(b) = \max_{i=1,\dots,k} (y^i)^\top b$ and define $Y := \text{conv}\{y^1, \dots, y^k\}$. Then $v(b) = \max_{y \in Y} b^\top y$. Since Y is a polyhedron admitting the representation $Y = \{y \mid A^\top y = c, y \geq 0\}$, linear programming duality gives $v(b) = \inf\{c^\top x \mid Ax \geq b\}$.

($\Leftarrow$) Let $v(b) = \inf\{c^\top x \mid Ax \geq b\}$. Dualising, $v(b) = \sup\{b^\top y \mid A^\top y = c, y \geq 0\}$. Since the feasible set $Y = \{y \mid A^\top y = c, y \geq 0\}$ is a polyhedron, the supremum is attained at its extreme points $y^1, \dots, y^k$, yielding $v(b) = \max_{i=1,\dots,k} (y^i)^\top b$. Thus v is piecewise affine convex. $\square$

The following proposition is the key technical lemma for the completeness proof of GLP. It states that two parametric linear programs with the same value function must share the same epigraph polyhedron, providing the bridge between semantic equality and polyhedral equality that the completeness argument requires.

Proposition 4.3. *Let*

$$v_i(y) := \inf\{1^\top x \mid A_i x \geq B_i y\}, \quad i = 1, 2,$$

where $A_i \in \mathbb{R}^{m_i \times n_i}$ and $B_i \in \mathbb{R}^{m_i \times k}$. If $v_1(y) = v_2(y)$ for all $y \in \mathbb{R}^k$, then both programs admit a common representation through the same epigraph polyhedron:

$$v_i(y) = \inf\{t \mid (y, t) \in E\}, \quad E := \text{epi}(v_1) = \text{epi}(v_2).$$

Proof. For each $i = 1, 2$, the epigraph of v_i satisfies

$$(y, t) \in \text{epi}(v_i) \iff \exists x : A_i x \geq B_i y, 1^\top x \leq t,$$

so $\text{epi}(v_i)$ is the projection onto (y, t) of the polyhedron $R_i = \{(x, y, t) \mid A_i x \geq B_i y, 1^\top x \leq t\}$. Projections of polyhedra are polyhedra, so each $\text{epi}(v_i)$ is a polyhedron. Since $v_1 = v_2$ pointwise, we have $\text{epi}(v_1) = \text{epi}(v_2) =: E$. Setting $E = \{(y, t) \mid Cy + dt \geq 0\}$, we obtain $v_i(y) = \inf\{t \mid Cy + dt \geq 0\}$ for both $i = 1, 2$. $\square$

With the equivalences above in place, we now define the semantic category of this chapter. **LPRel** describes morphisms as parametric linear programs; by the propositions above, these coincide with polyhedral functions, and we will use both descriptions interchangeably according to which is more convenient.

Definition 4.4 (The prop **LPRel** of Linear Programs). *The prop **LPRel** is defined as follows:*

- *Objects are natural numbers $n \in \mathbb{N}$, interpreted as the space $\mathbb{R}^n$.*
- *A morphism $n \rightarrow m$ is a right-hand side parametric linear program with value function*

$$(x, y) \mapsto \inf_z \{c^\top z \mid Az \geq B[x \ y]^\top + b\},$$

where $x \in \mathbb{R}^n$ and $y \in \mathbb{R}^m$.

- *Composition is given by minimisation over the shared variable.*
- *The monoidal product on objects is addition $n \otimes m := n + m$, and on morphisms is direct sum.*

4.2 SYNTAX: THE ALGEBRA GLP

With the semantic equivalences established, we now introduce the final graphical algebra of the thesis. The algebra **GLP** extends **GPA** with a single new generator whose semantics encodes the linear objective function $c^\top x$. Just as the gray arrow in **GQA** lifted the theory from binary relations to weighted relations by "measuring" the ℓ_2 norm, this new generator lifts **GPA** from polyhedral constraints to full linear programs by attaching a linear cost to the feasible region.

Definition 4.5 (GLP). *We call **GLP** the symmetric monoidal theory obtained by extending **GPA** with the following generator:*

$$\llbracket \square - \rrbracket = (\bullet, x) \mapsto x$$

together with the following axioms governing its interaction with the existing generators:

$$\begin{array}{lll}
 \text{(linearity)} & \begin{array}{c} \square - \\ \square - \end{array} & = \begin{array}{c} \square - \end{array} \begin{array}{c} \bigcirc \\ \bigcirc \end{array} \\
 \text{(zeroeq)} & \begin{array}{c} \square - \end{array} \begin{array}{c} \bigcirc \end{array} & = \begin{array}{c} \square \end{array} \\
 \text{(inf)} & \begin{array}{c} \square - \end{array} \begin{array}{c} \boxed{\geq} - \end{array} & = \begin{array}{c} \square - \end{array}
 \end{array}$$

The axioms represent expected properties of the linear cost function. The one on the left express the linear property $f(x) + f(y) = f(x + y)$. Observe that f is the interpretation of the $\square -$ and the sum denotes the vertical composition; composing two $\square -$ vertically (adding) is the same as adding the wires in a single $\square -$. This axiom is also the fundamental property of LPs, which is the fact that they are invariant to column stochastic matrices (see 4.1). In turn,

the equation on the right express the fundamental property of minimization, the infimum of any quantity lies in its lower bound.

$$\inf_x \{x \mid x \geq y\} = y$$

Finally, the axiom at the middle is the tautological case.

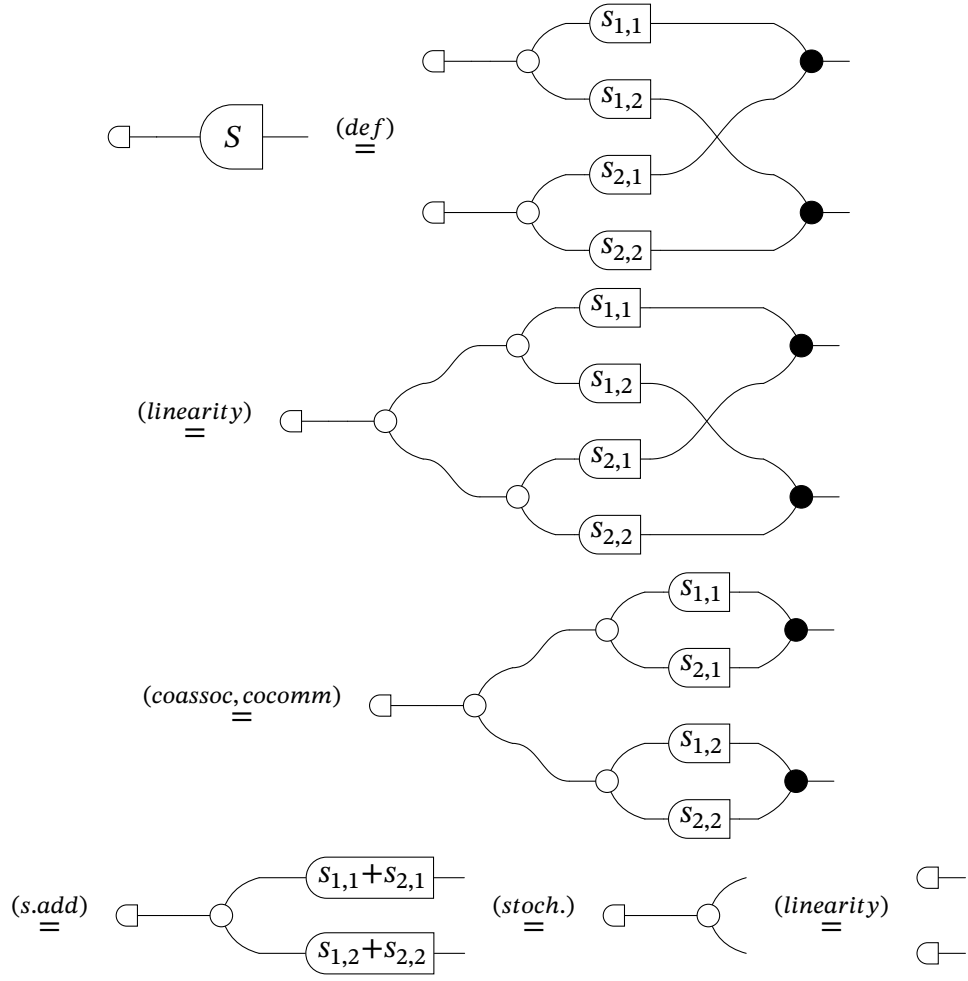
$$\inf 0 = 0$$

From this simple axioms, we get immediatly another core property of linear programs, namely, that they are invariant to stochastic matrices.

Remark 4.1 (Invariance under column-stochastic matrices). *The linearity axiom encodes the fundamental invariance of linear programs under column-stochastic reweightings of the objective: for any $n \times m$ matrix S whose columns each sum to one,*

$$\Box^{\otimes n}; S^{op} = \Box^{\otimes m},$$

where S^{op} denotes the comatrix of S . Note that only the column sums are used; non-negativity of the entries plays no role. The proof is diagrammatic, we display the proof for a 2×2 matrix S with $s_{1,1} + s_{2,1} = s_{1,2} + s_{2,2} = 1$; every step is an instance of an axiom scheme that applies wire-by-wire, so the same chain of rewrites goes through verbatim for arbitrary $n \times m$.



Such result is particularly interesting because it follows a pattern present in every algebra presented so far. In the digrammatic calculi of Linear subspaces (GLA) we can demonstrate that surjective matrices are very special because the $\bullet$ is invariant to it, which is a very important feature in the completeness of the language itself. The same occurs in GPA, where non-negative matrices slide through the $\begin{smallmatrix} \top \\ \geq \end{smallmatrix}$. Here, the new class of matrices that interact with the new generator $\square$ is that of stochastic matrices. Other works have spotted the same pattern, such as [Ste+24] which shows that a generator representing the l_2 norm is invariant to orthogonal matrices (also are normal distributed random variables).

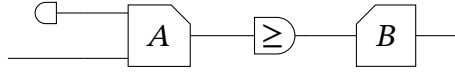
Is not yet entirely clear the relationship between each class of morphisms and that class of matrices and how they emerge. However is a signature and very interesting property of such languages.

With the SMT above we now define the complete category.

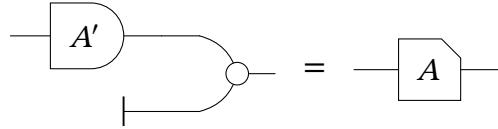
Definition 4.6. We call $\text{FreeP}_{\text{GLP}}$ the PROP freely generated by GLP.

The normal form theorem for GLP extends the one for GPA: every diagram can be reduced to a GPA block encoding the constraints, composed with the new generator encoding the objective. The proof proceeds by structural induction, using the existing normal form for GPA as the inductive base.

Theorem 4.1 (Normal Form). *For every diagram $c \in \mathbf{GLP}$, there exists a diagram c' in normal form such that $c = c'$:*

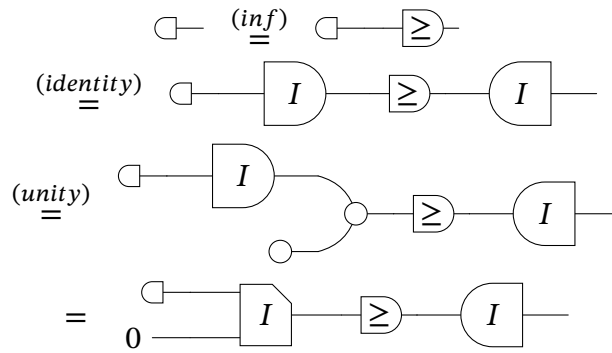


With A and b being an affine transformation. Namely:

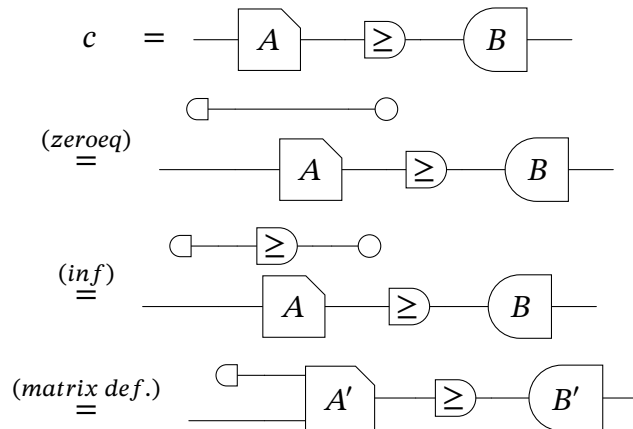


Proof. The proof is by structural induction. First we show the two base cases:

- The generator $\sqsubset -$

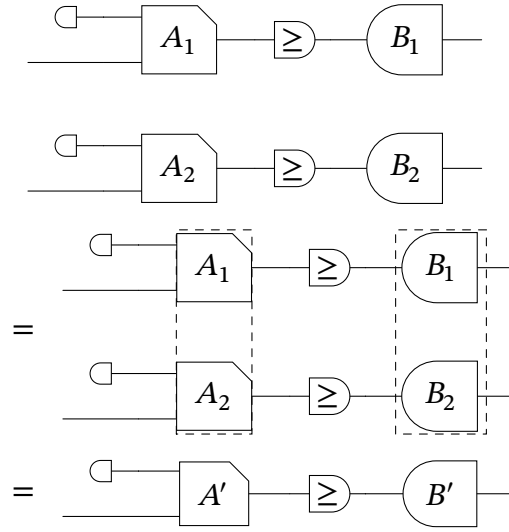


- A diagram $c \in \mathbf{GPA}$

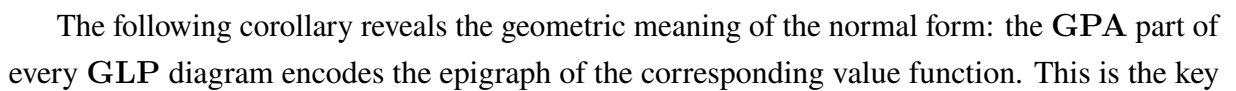


For the inductive step, we verify that for any two diagrams $c, d \in \mathbf{GLP}$ satisfying the inductive hypothesis, both horizontal and vertical composition preserve the normal form. Closure under

vertical composition follows by stacking the two normal forms and collapsing the two independent A_i , ge , B_i blocks into a single block-diagonal matrix pair, as shown below:



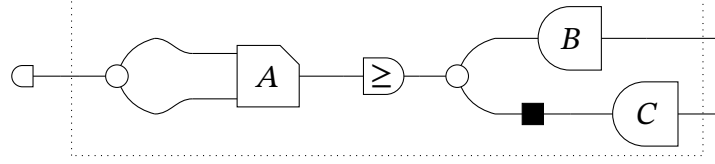
For horizontal composition, the two normal forms are joined along the wire connecting B_1 to A_2 ; applying Fourier-Motzkin Elimination to this pair recovers a fresh matrix-ge-comatrix triple, after which the two resulting runs of matrices, one in each direction, compose into a single matrix and a single comatrix, recovering the normal form:



The following corollary reveals the geometric meaning of the normal form: the **GPA** part of every **GLP** diagram encodes the epigraph of the corresponding value function. This is the key

structural insight connecting the syntax to the semantic equivalence of Proposition 4.3.

Corollary 4.1 (Epigraph Form). *Given a diagram $d \in \text{GLP}$ in normal form, it can always be rewritten as a diagram d' of the form:*



where the semantics of the dotted diagram is

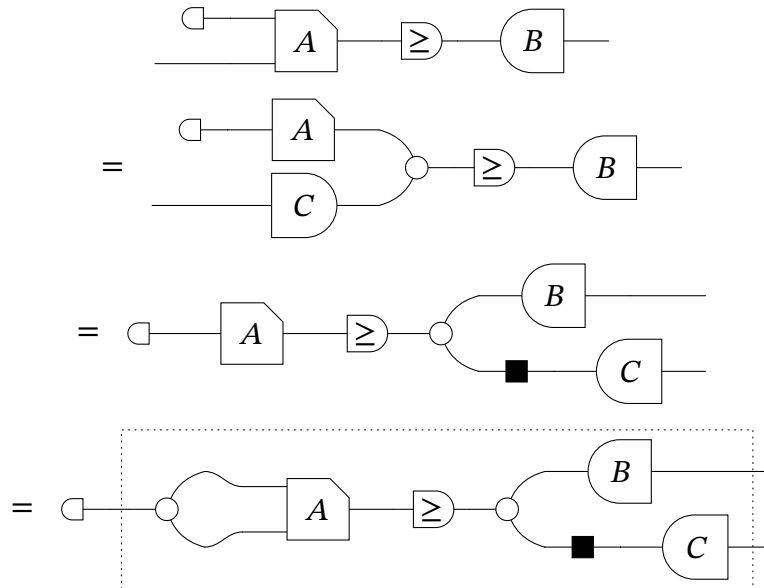
$$\left\{ (t, y) \mid \exists x, A'x + a \geq [B \ - C]y, t \geq x, y = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \right\}$$

. This is the epigraph of the original diagram d

$$\begin{aligned} [d](y_1, y_2) &= \inf_z z \quad \text{s.t. } Az + a \geq By_1 - Cy_2 \\ \text{epi}([d]) &= \{(t, y_1, y_2) \mid \exists z \leq t, Az + a \geq By_1 - Cy_2\} \end{aligned}$$

such that $A = [A' \ C]$. We call this the epigraph form of a linear program.

Proof. Starting from d in normal form, split the relation generator along $A = [A' \ C]$ and pull the resulting C -block through the $\geq$ generator using the antipode axiom:



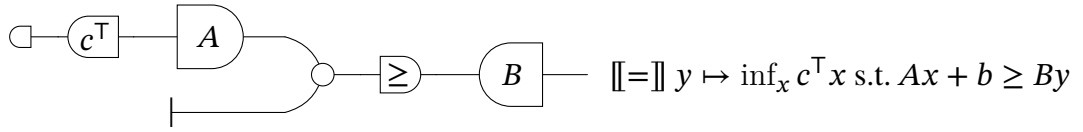
The resulting diagram is precisely the claimed form d' , with the dotted box exhibiting $\text{epi}([d])$ as a GPA diagram. $\square$

4.3 PRESENTATION RESULTS FOR LPREL

With the syntax and its normal form in place, we are now ready to prove the three presentation properties for GLP.

Lemma 4.1 (Expressivity). *For every $f : m \rightarrow n$ in **LPRel** there exists a Σ -term $s : m \rightarrow n$ in $\text{Hom}_{\text{FreeP}_{\text{GLP}}}(m, n)$ such that $[s] = f$.*

Proof. Any RHS Parametric linear program can be written as $\inf_z \{c^\top z \mid Ax + b \geq By\}$ therefore is enough to show that we can always depict such a problem. The following diagram provides the required syntactic representative:



□

Lemma 4.2 (Soundness). *If $s = t$ in $\text{FreeP}_{\text{GLP}}$, then $[s] = [t]$ in **LPRel**.*

Proof. All axioms inherited from **GPA** are already sound by the results of the previous section. It suffices to verify that the new axioms involving the LP generator preserve the semantics, which follows by direct computation:

$$\begin{aligned}
 (\text{linear.ge}) \quad & \Box \vdash \llbracket = \rrbracket (\bullet, y) \mapsto y = (\bullet, y) \mapsto \inf_{x \geq y} x \quad \llbracket = \rrbracket \quad \Box \vdash \geq \\
 (\text{lin.sum}) \quad & \Box \vdash \llbracket = \rrbracket (\bullet, (y_1, y_2)) \mapsto y_1 + y_2 \quad \llbracket = \rrbracket \quad \Box \vdash \bigcirc \\
 (\text{lin.zero}) \quad & \Box \vdash \llbracket = \rrbracket (\bullet, \bullet) \mapsto \inf_{x=0} x = 0 \quad \llbracket = \rrbracket \quad \Box \vdash \square
 \end{aligned}$$

□

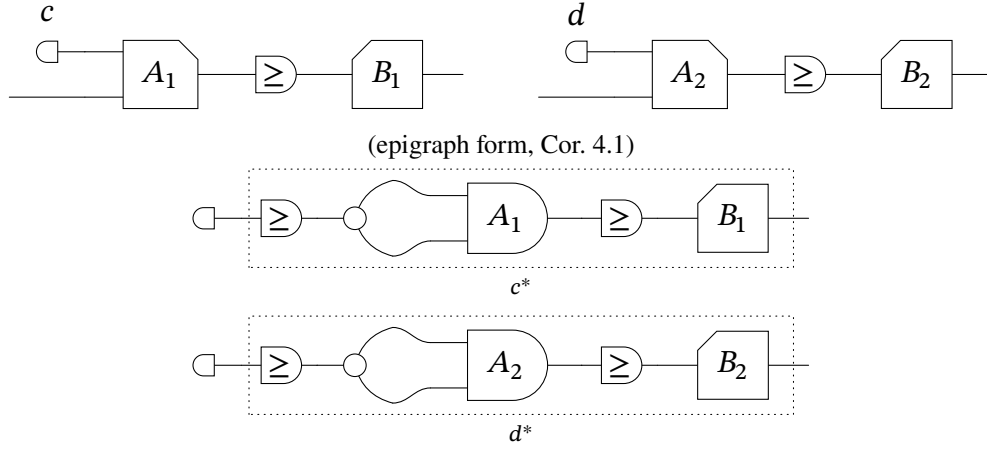
The final result of the thesis is the completeness theorem for GLP. The proof brings together all the main tools developed throughout: the epigraph form of Corollary 4.1, the shared epigraph property of Proposition 4.3, the completeness of GPA, and the FME-based normal form.

Lemma 4.3 (Completeness). *If $[c] = [d]$ in **LPRel**, then $c = d$ in $\text{FreeP}_{\text{GLP}}$.*

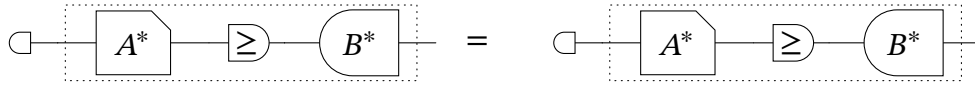
Proof. Let $v_1 = [c]$ and $v_2 = [d]$. By assumption $v_1 = v_2$, so Proposition 4.3 gives $\text{epi}(v_1) = \text{epi}(v_2) =: E$.

By Corollary 4.1, both c and d admit epigraph forms $c = \Box \vdash -; c^*$ and $d = \Box \vdash -; d^*$ with $[c^*] = [d^*] = E$ in **PolyRel**. Since c^* and d^* are GPA diagrams with the same denotation, completeness of GPA gives $c^* = d^*$ in $\text{FreeP}_{\text{GPA}}$; in particular both reduce to the same GPA

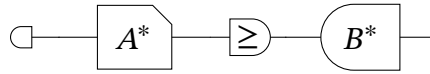
normal form. Composing this common diagram with the leftover $\sqsubset$ – collapses c and d to the same **GLP** diagram:



(the dotted boxes c^*, d^* are **GPA** diagrams with $[c^*] = [d^*] = E$; by **GPA** completeness both reduce to the same normal form)



(composing the common **GPA** normal form with the leftover $\sqsubset$ yields the same **GLP** diagram)



Therefore $c = \sqsubset -$; $c^* = \sqsubset -$; $d^* = d$. □

As a consequence of the three lemmas above:

Theorem 4.2. *LPRel is presented by $\text{FreeP}_{\text{GLP}}$.*

Proof. By the preceding three lemmas. □

Finally, we've built, through the addition of one generator and three axioms, an elegant and complete diagrammatic theory for parametric linear programming. This result does more than unlock a new algebraic and computational way to interact with LP problems: it relocates linear programming itself. A linear program is ordinarily presented as a function to be evaluated, an $x \mapsto c^\top x$ subject to constraints, solved from the outside by an algorithm. Here it is instead a morphism, an object with an interface that composes with others of its kind, and solving it, evaluating it, and eliminating one of its variables are all the same operation, composition in **LPRel**. Optimization stops being a technique applied to a relational structure and becomes a property internal to it, and linear programming, in particular, is lifted out of the setting of functional thinking and given a new home as a relational field embedded in the Extended-Lawvere Quantale category.

The **GLP** theory, in turn, has a natural extension to an even more expressive theory when enriched with the gray arrow generator from **GQA** [Ste+24]. We believe that the class of

quadratic programs could be, just as was done in this work, fully axiomatized by such broader SMIT. However, such studies are left for future works, along with other possible applications of current results.

5 CONCLUSION

We started with a remind of the quantale-valued relations and the quiet observation that composing them is already a form of optimization, transitivity being its simplest instance. From there, we recalled and refined the graphical theory of polyhedral relations, giving it a home inside this same categorical hierarchy rather than treating it as a self-contained formalism. On that foundation, we built **GLP**, a graphical algebra for parametric linear programs: a single new generator for the linear objective, a handful of axioms, and a proof that these suffice, both for completeness and for a normal form (Theorem 4.1).

Three results are worth calling out on their own. First, **GLP** is *complete*: any two diagrams with the same semantics can be rewritten into one another using only the axioms of the algebra, so there is no gap between what the theory can prove equal and what is actually equal. Second, every such diagram admits a *normal form* (Theorem 4.1), a canonical shape that every parametric linear program reduces to, which is what makes the correspondence with value functions below more than a loose analogy. Third, one of the axioms encodes something genuinely structural about linear programming rather than something we imposed on it: the *invariance of a linear program under column-stochastic reweighting of its objective* (Remark 4.1) falls directly out of the algebra, as a single generator-level equation, instead of being proved separately for each formulation. This is one of the core properties of linear systems and thanks to our Graphical Algebra is a syntactical operation in itself. As we claimend at the introduction, the diagrammatic calculi makes the system properties evident in the syntax, instead of a higher level result and this is the evidence.

A linear program is now a morphism like any other in the hierarchy, one that composes with the relations and quadratic blocks studied earlier without any special pleading. Its diagrammatic syntax, once reduced to normal form, coincides with the syntax of its own value function, so the diagram and the object it computes are, in a precise sense, the same picture drawn twice.

Just like that, other theories can be developed by enriching the here presented language. Figure 5.1 lays out the resulting hierarchy of categorical structures that were developed by this very recipe and what can actually be achieved in the end of the road.

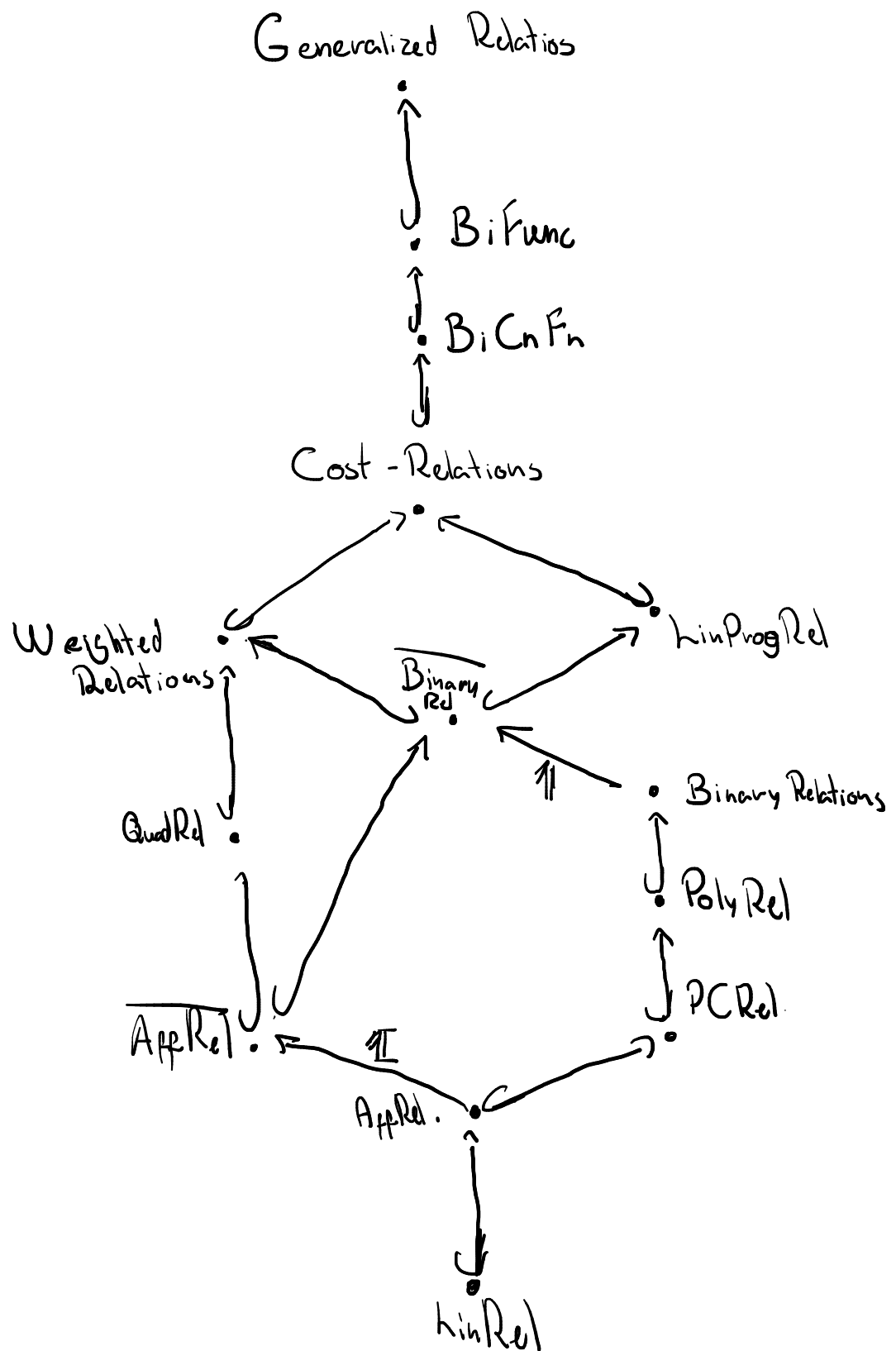


Figure 5.1 – The hierarchy of relational categories developed in this thesis. Each inclusion preserves the monoidal and compositional structure of the layer below.

- Moving upward corresponds to increasing expressive power: each layer generalises the one below by enlarging the class of morphisms, while preserving the compositional structure.
- The morphisms marked $\hookrightarrow$ are inclusion functors, embedding each category as a full subcategory of the one above.
- The morphism marked **1** denotes the indicator function embedding, which sends a set-valued relation to its $\{0, +\infty\}$ -valued characteristic function.

It is a modest step, in the end, toward something larger: a diagrammatic and more calculational language expressive enough to hold optimization in its own right. The theories presented and developed here read less like separate constructions sharing a notation and one theory examined at different levels of resolution. A sequential increment of the theory, one generator at a time, lifting the whole theory.

STATEMENT ON THE USE OF ARTIFICIAL INTELLIGENCE

The use of AI in this paper in a few different ways. When it comes to the writing of the thesis it was restricted to the following use:

- Tikz Writing: Convert tables and diagrams from handwritten images to tikz syntax.
- Writing review: AI searched for grammatical inconsistencies, typos, open-ended and poorly-formed sentences.
- Review the mathematical argument: The AI was instructed to look for loops or failed arguments across mathematical proofs and derivations.
- Loop Review: Read and evaluate the work multiple times and provide critics and feedback about the overall writing.
- Latex Formatting: AI was used to adapt the paper to the necessary standards demanded by a dissertation. This refers only to document structure(font size, title format, citations etc) not the writing content.

AI most relevant effort came in into the appendix for helping writing down a long list of well established mathematical definitions to provide the reader the necessary background. The main body of the work (from introduction to conclusion) is almost entirely written by hand having AI only as a default reviewer.

During the development and research of the thesis AI was used as a learning gate and searching mechanism to discuss ideas, look for references, investigate topics and understand papers and works from others.

Bibliography

- [BDS21] Filippo Bonchi, Alessandro Di Giorgio, and Paweł Sobociński. “Diagrammatic Polyhedral Algebra”. In: *41st IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2021)*. Vol. 213. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 40:1–40:18. DOI: 10.4230/LIPIcs.FSTTCS.2021.40. arXiv: 2105.10946.
- [BDZ21] Filippo Bonchi, Alessandro Di Giorgio, and Fabio Zanasi. “From Farkas’ Lemma to Linear Programming: an Exercise in Diagrammatic Algebra”. In: *9th Conference on Algebra and Coalgebra in Computer Science (CALCO 2021)*. Ed. by Fabio Gadducci and Alexandra Silva. Vol. 211. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 9:1–9:19. ISBN: 978-3-95977-212-9. DOI: 10.4230/LIPIcs.CALCO.2021.9. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CALCO.2021.9>.
- [BF18] John C. Baez and Brendan Fong. “A Compositional Framework for Passive Linear Networks”. In: *Theory and Applications of Categories* 33.38 (2018), pp. 1158–1222. arXiv: 1504.05625. URL: <http://www.tac.mta.ca/tac/volumes/33/38/33-38.pdf>.
- [Bon+19] Filippo Bonchi et al. “Graphical Affine Algebra”. In: *Proceedings of the 34th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2019)*. Vancouver, BC, Canada: IEEE, 2019, pp. 1–12. DOI: 10.1109/LICS.2019.8785877.
- [BSZ17] Filippo Bonchi, Paweł Sobociński, and Fabio Zanasi. “Interacting Hopf Algebras”. In: *Journal of Pure and Applied Algebra* 221.1 (2017). Original complete axiomatisation of Graphical Linear Algebra, pp. 144–184. ISSN: 0022-4049. DOI: 10.1016/j.jpaa.2016.06.002. arXiv: 1403.7048.
- [CD11] Bob Coecke and Ross Duncan. “Interacting Quantum Observables: Categorical Algebra and Diagrammatics”. In: *New Journal of Physics* 13 (2011), p. 043016. DOI: 10.1088/1367-2630/13/4/043016. arXiv: 0906.4725.
- [Coe+18] Bob Coecke et al. “Generalized Relations in Linguistics & Cognition”. In: *Theoretical Computer Science* (2018). Earlier conference version in WoLLIC 2017, LNCS vol. 10388, DOI: 10.1007/978-3-662-55386-2_18. ISSN: 0304-3975. DOI: 10.1016/j.tcs.2018.03.008. URL: <https://www.sciencedirect.com/science/article/pii/S0304397518301476>.

- [Fre+25] Iago Leal de Freitas et al. *Generalizing the Invertible Matrix Theorem with Linear Relations using Graphical Linear Algebra*. 2025. arXiv: 2502.16783 [cs.SC]. URL: <https://arxiv.org/abs/2502.16783>.
- [FS19] Brendan Fong and David I. Spivak. “Hypergraph Categories”. In: *Journal of Pure and Applied Algebra* 223.11 (2019), pp. 4746–4777. ISSN: 0022-4049. DOI: 10.1016/j.jpaa.2019.02.014. arXiv: 1806.08304.
- [Han+24] Tyler Hanks et al. “A Compositional Framework for First-Order Optimization”. In: *arXiv preprint* (2024). arXiv: 2403.05711. URL: <https://arxiv.org/abs/2403.05711>.
- [JS91] André Joyal and Ross Street. “The Geometry of Tensor Calculus, I”. In: *Advances in Mathematics* 88.1 (1991), pp. 55–112. ISSN: 0001-8708. DOI: 10.1016/0001-8708(91)90003-P. URL: <https://www.sciencedirect.com/science/article/pii/000187089190003P>.
- [Kad26] Fethi Kadhi. “BRIDGING CATEGORY THEORY AND OPTIMIZATION: A CATEGORICAL CHARACTERIZATION OF CONVEX ENVELOPES”. In: *Journal of Mathematical Sciences* (Feb. 2026), pp. 1–16. DOI: 10.1007/s10958-026-08222-8.
- [Lan98] Saunders Mac Lane. *Categories for the Working Mathematician*. 2nd. Vol. 5. Graduate Texts in Mathematics. Springer, New York, 1998. ISBN: 978-0-387-98403-2. DOI: 10.1007/978-1-4757-4721-8.
- [Law73] F. William Lawvere. “Metric Spaces, Generalized Logic, and Closed Categories”. In: *Rendiconti del Seminario Matematico e Fisico di Milano* 43.1 (1973). Reprinted in: *Reprints in Theory and Applications of Categories*, No. 1 (2002), pp. 1–37, pp. 135–166. DOI: 10.1007/BF02924844.
- [PRS22] João Paixão, Lucas Rufino, and Paweł Sobociński. “High-level axioms for graphical linear algebra”. In: *Science of Computer Programming* 218 (2022), p. 102791. ISSN: 0167-6423. DOI: 10.1016/j.scico.2022.102791. URL: <https://www.sciencedirect.com/science/article/pii/S0167642322000247>.
- [PZ23] Robin Piedeleu and Fabio Zanasi. *An Introduction to String Diagrams for Computer Scientists*. 2023. arXiv: 2305.08768 [cs.LO]. URL: <https://arxiv.org/abs/2305.08768>.
- [Roc70] R. Tyrrell Rockafellar. *Convex Analysis*. Vol. 28. Princeton Mathematical Series. Princeton, New Jersey: Princeton University Press, 1970. ISBN: 978-0-691-01586-6. DOI: 10.1515/9781400873173.

- [Sel10] Peter Selinger. “A Survey of Graphical Languages for Monoidal Categories”. In: *New Structures for Physics*. Ed. by Bob Coecke. Vol. 813. Lecture Notes in Physics. Springer, Berlin, Heidelberg, 2010, pp. 289–355. DOI: 10.1007/978-3-642-12821-9_4. arXiv: 0908.3347.
- [SS21] Dario Stein and Sam Staton. “Compositional Semantics for Probabilistic Programs with Exact Conditioning”. In: *2021 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. 2021, pp. 1–13. DOI: 10.1109/LICS52264.2021.9470552.
- [SS24] Dario Stein and Richard Samuelson. “Towards a Compositional Framework for Convex Analysis (with Applications to Probability Theory)”. In: *Foundations of Software Science and Computation Structures (FoSSaCS 2024)*. Vol. 14574. Lecture Notes in Computer Science. Springer, Cham, 2024, ? DOI: 10.1007/978-3-031-57228-9_9. arXiv: 2312.02291.
- [Ste+24] Dario Stein et al. “Graphical Quadratic Algebra”. In: *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2024)*. Springer, Cham, 2024. DOI: 10.1007/978-3-032-11176-0_18. arXiv: 2403.02284.
- [Wil15] Simon Willerton. *The Legendre-Fenchel Transform from a Category Theoretic Perspective*. 2015. arXiv: 1501.03791. URL: <https://arxiv.org/abs/1501.03791>.
- [Win93] Glynn Winskel. *The Formal Semantics of Programming Languages: An Introduction*. Foundations of Computing. Cambridge, Massachusetts: MIT Press, 1993. ISBN: 978-0-262-23169-5. URL: <https://mitpress.mit.edu/9780262731034/>.

A CATEGORICAL AND GRAPHICAL LINEAR ALGEBRA

This appendix develops the categorical background used by the graphical calculi in this thesis. Its purpose is not to survey category theory in full, but to make the passage from ordinary composition to Graphical Linear Algebra (GLA) explicit and self-contained. The central idea is simple: a diagram is a morphism, joining diagrams eliminates their common variables, and placing diagrams side by side forms independent systems. Category theory isolates these two operations; symmetric monoidal categories make them graphical; and GLA specializes the resulting language to systems of linear equations.

The development proceeds in four stages. First we recall categories, functors, and the category of relations. We then add a symmetric monoidal product and explain the corresponding string-diagram notation. Next we introduce PROPs, monoid and Frobenius structures, which provide the algebraic operations for copying, deleting, adding, and constraining variables. Finally we define the PROP of linear relations and state the presentation theorem identifying it with GLA.

A.1 CATEGORIES: SYSTEMS AND THEIR COMPOSITION

Definition A.1 (Category). A category $\mathcal{C}$ consists of:

- a class of objects $A, B, \dots$;
- for each pair of objects, a class $\mathcal{C}(A, B)$ of morphisms $f : A \rightarrow B$;
- for every A , an identity morphism $\text{id}_A : A \rightarrow A$;
- for $f : A \rightarrow B$ and $g : B \rightarrow C$, a composite $g \circ f : A \rightarrow C$.

Composition is associative and identities are neutral:

$$h \circ (g \circ f) = (h \circ g) \circ f, \quad \text{id}_B \circ f = f = f \circ \text{id}_A.$$

A category therefore retains only what is required to connect processes. An object records the type of an interface, while a morphism records a process with a specified input and output. The equation $g \circ f$ is meaningful exactly when the output type of f agrees with the input type of g . Associativity says that a composite process does not depend on how it is parenthesized.

Two examples will be used repeatedly.

Example A.1 (Functions). *The category $\mathbf{Set}$ has sets as objects and functions as morphisms. Composition and identities are the usual ones. A function is deterministic: each input has exactly one output.*

Example A.2 (Relations). *The category $\mathbf{Rel}$ has sets as objects and binary relations $R \subseteq X \times Y$ as morphisms $R : X \rightarrow Y$. Given $R : X \rightarrow Y$ and $S : Y \rightarrow Z$, their composite is*

$$S \circ R = \{(x, z) \mid \text{there exists } y \in Y \text{ with } xRy \text{ and } ySz\}.$$

The identity on X is the equality relation $\{(x, x) \mid x \in X\}$. Thus relational composition eliminates the shared variable y . Unlike a function, a relation may associate an input with no output or with several outputs; this is why relations, rather than functions, are the appropriate semantics for systems of equations and inequalities.

Definition A.2 (Functor). *A functor $F : \mathcal{C} \rightarrow \mathcal{D}$ assigns an object FA to every object A and a morphism $Ff : FA \rightarrow FB$ to every morphism $f : A \rightarrow B$, while preserving identities and composition:*

$$F(\text{id}_A) = \text{id}_{FA}, \quad F(g \circ f) = Fg \circ Ff.$$

It is faithful when every map $\mathcal{C}(A, B) \rightarrow \mathcal{D}(FA, FB)$ is injective, and full when every such map is surjective.

In this work, a functor usually interprets syntax as semantics. Faithfulness means that semantically equal diagrams are already equal in the calculus; fullness means that every semantic process of the intended kind can be drawn. These are the categorical forms of completeness and expressivity, respectively.

A.2 PARALLEL COMPOSITION AND STRING DIAGRAMS

Ordinary categorical composition describes processes in sequence. To describe independent processes in parallel we need a second operation.

Definition A.3 (Symmetric monoidal category). *A symmetric monoidal category is a category $\mathcal{C}$ equipped with a bifunctor*

$$\otimes : \mathcal{C} \times \mathcal{C} \longrightarrow \mathcal{C},$$

a monoidal unit I , and coherent natural isomorphisms for associativity, units, and symmetry. In particular, for morphisms $f : A \rightarrow B$ and $g : C \rightarrow D$ there is a morphism

$$f \otimes g : A \otimes C \longrightarrow B \otimes D,$$

and the interchange law holds:

$$(f_2 \circ f_1) \otimes (g_2 \circ g_1) = (f_2 \otimes g_2) \circ (f_1 \otimes g_1).$$

The symmetry $\sigma_{A,B} : A \otimes B \rightarrow B \otimes A$ exchanges the two components.

Mac Lane’s coherence theorem allows the associativity and unit isomorphisms to be suppressed without ambiguity [Lan98]. We use this strict notation throughout.

A symmetric monoidal category has a sound two-dimensional notation: its *string diagrams* [JS91; Sel10]. A wire represents an object and a box with input and output wires represents a morphism. Reading from left to right, joining the outputs of f to the inputs of g represents $g \circ f$; placing f above g represents $f \otimes g$. A bare wire is an identity and crossing wires represents symmetry. The interchange law is built into the geometry: disjoint boxes may move past one another because their relative horizontal order carries no information. Consequently, diagrams are considered equal up to deformations that preserve connectivity, boundary, and the order of ports on each box.

$$\begin{array}{c}
 \begin{array}{c} \boxed{f} \quad \boxed{g} \\ \hline \end{array} \\
 (g \circ f) \otimes (k \circ h) \\
 \begin{array}{c} \boxed{h} \quad \boxed{k} \\ \hline \end{array}
 \end{array}
 =
 \begin{array}{c}
 \begin{array}{c} \boxed{f} \quad \boxed{g} \\ \hline \end{array} \\
 (g \otimes k) \circ (f \otimes h) \\
 \begin{array}{c} \boxed{h} \quad \boxed{k} \\ \hline \end{array}
 \end{array}$$

Figure A.1 – The interchange law expressed as a string diagram.

For **Rel** the monoidal product is Cartesian product on objects. If $R : X \rightarrow Y$ and $S : U \rightarrow V$, then

$$R \otimes S : X \times U \longrightarrow Y \times V, \quad ((x, u), (y, v)) \in R \otimes S \iff xRy \text{ and } uSv.$$

The singleton set is the unit. Sequential composition eliminates shared variables; monoidal composition juxtaposes independent systems.

A.3 PROPS AND GRAPHICAL THEORIES

The graphical calculi used in this thesis have one generating wire type. An interface is therefore determined only by its number of wires.

Definition A.4 (PROP). A PROP is a strict symmetric monoidal category whose objects are the natural numbers, whose tensor product on objects is addition, and whose monoidal unit is 0. A morphism $m \rightarrow n$ is drawn with m input wires and n output wires.

Thus 1 is the basic wire, $n = 1^{\otimes n}$ is a bundle of n wires, and $m \otimes n = m + n$. Matrices provide the standard example: for a field $\mathbb{K}$, let $\text{Mat}_{\mathbb{K}}$ have morphisms $m \rightarrow n$ given by $n \times m$ matrices. Composition is matrix multiplication, tensor is block diagonal sum, and the symmetry is a permutation matrix.

Definition A.5 (Symmetric monoidal theory). A symmetric monoidal theory (SMT) is a pair (Σ, E) , where Σ is a collection of typed generators $o : m \rightarrow n$ and E is a collection of equations between equally typed string diagrams built from those generators, identities, symmetries, composition, and tensor product. The PROP presented by the theory is denoted $\mathbf{P}_{\Sigma, E}$.

The word ‘‘free’’ in a free PROP means that two generated diagrams are equal only when this follows from the symmetric monoidal laws and the equations in E . To give a semantics for the syntax is therefore to give a strict symmetric monoidal functor

$$\llbracket - \rrbracket : \mathbf{P}_{\Sigma, E} \longrightarrow \mathbf{Sem}$$

that sends every generating equation to a true semantic equality.

Definition A.6 (Soundness, completeness, and expressivity). *Let $c, d : m \rightarrow n$ be diagrams.*

- *The theory is sound when $c =_E d$ implies $\llbracket c \rrbracket = \llbracket d \rrbracket$.*
- *It is complete when $\llbracket c \rrbracket = \llbracket d \rrbracket$ implies $c =_E d$.*
- *It is expressive when every morphism of the semantic PROP is the interpretation of some diagram.*

Equivalently, the interpretation functor is well defined, faithful, and full, respectively.

These distinctions are essential. Soundness prevents an equation from identifying different systems. Completeness ensures that diagram rewriting can prove every valid semantic equality. Expressivity ensures that the language can denote every system under consideration.

A.4 MONOIDS, COMONOIDS, AND FROBENIUS STRUCTURE

The nodes in GLA are not arbitrary operations. They arise from standard algebraic structures internal to a symmetric monoidal category.

Definition A.7 (Monoid and comonoid). *A commutative monoid on an object A consists of $\mu : A \otimes A \rightarrow A$ and $\eta : I \rightarrow A$ satisfying associativity, unitality, and commutativity. Dually, a cocommutative comonoid consists of $\delta : A \rightarrow A \otimes A$ and $\varepsilon : A \rightarrow I$ satisfying coassociativity, counitality, and cocommutativity.*

On a vector space, the intended monoid is addition: $\mu(x_1, x_2) = x_1 + x_2$ and $\eta(*) = 0$. The intended comonoid copies and discards data: $\delta(x) = (x, x)$ and $\varepsilon(x) = *$. Copying is linear, but its converse is not a function; this is the first indication that linear relations are the natural ambient category.

Definition A.8 (Special commutative Frobenius algebra). *A commutative monoid (μ, η) and a cocommutative comonoid (δ, ε) on the same object form a special commutative Frobenius algebra when*

$$(\mu \otimes \text{id}) \circ (\text{id} \otimes \delta) = \delta \circ \mu = (\text{id} \otimes \mu) \circ (\delta \otimes \text{id})$$

and $\mu \circ \delta = \text{id}$.

The Frobenius and special laws imply the *spider theorem*: every connected diagram built only from $\mu, \eta, \delta, \varepsilon$ is determined by its number of input and output wires. It may therefore be contracted to one multi-input, multi-output node. This is the algebraic reason that wires can be split, merged, bent, and reconnected without recording an arbitrary parse tree.

The cup $\delta \circ \eta : I \rightarrow A \otimes A$ and cap $\varepsilon \circ \mu : A \otimes A \rightarrow I$ induced by a special commutative Frobenius algebra satisfy the snake equations. They make A self-dual and allow an input wire to be bent into an output wire, or conversely. A symmetric monoidal category equipped coherently with such a structure on each object is called a *hypergraph category* [FS19]. This compact structure is what makes the converse of a relation available graphically by reflection.

A.5 THE PROP OF LINEAR RELATIONS

Fix a field $\mathbb{K}$; all applications in this thesis use $\mathbb{K} = \mathbb{R}$.

Definition A.9 (Linear relation). A linear relation $R : m \rightarrow n$ is a linear subspace

$$R \subseteq \mathbb{K}^m \oplus \mathbb{K}^n.$$

We write $x R y$ when $(x, y) \in R$. The PROP $\mathbf{LinRel}_{\mathbb{K}}$ has natural numbers as objects and linear relations as morphisms. Its identity, composition, and tensor are

$$\begin{aligned} \text{id}_n &= \{(x, x) \mid x \in \mathbb{K}^n\}, \\ S \circ R &= \{(x, z) \mid \exists y \in \mathbb{K}^n : x R y \text{ and } y S z\}, \\ R \otimes S &= \{((x, u), (y, v)) \mid x R y, u S v\}. \end{aligned}$$

After the canonical identification $\mathbb{K}^m \oplus \mathbb{K}^p \cong \mathbb{K}^{m+p}$, the tensor on objects is addition.

The composite is again a linear subspace: the simultaneous constraints $(x, y) \in R$ and $(y, z) \in S$ define a linear subspace, and projecting away y preserves linearity. Hence composition of linear relations is exactly linear variable elimination.

Every matrix $A \in \mathbb{K}^{n \times m}$ determines the functional linear relation

$$\text{graph}(A) = \{(x, y) \in \mathbb{K}^m \oplus \mathbb{K}^n \mid y = Ax\}.$$

Matrix multiplication is recovered by relational composition:

$$\text{graph}(B) \circ \text{graph}(A) = \text{graph}(BA).$$

Linear relations are nevertheless strictly more general than matrices. For example, the converse

$$\text{graph}(A)^{\text{op}} = \{(y, x) \mid y = Ax\}$$

may be partial or multivalued. More generally, every linear relation admits a presentation by homogeneous linear equations:

$$R = \{(x, y) \mid Ex = Fy\}$$

for suitable matrices E and F . This cospan form is the semantic origin of GLA normal forms.

A.6 GRAPHICAL LINEAR ALGEBRA

Graphical Linear Algebra is a graphical presentation of $\mathbf{LinRel}_{\mathbb{K}}$ [BSZ17; PRS22]. Its diagrams are generated by two interacting families of nodes and by one scalar box for each $k \in \mathbb{K}$. We read diagrams from left to right. The following table gives the type and exact relation denoted by every basic orientation; the reversed orientation denotes the converse relation.

Table A.1 – Generators of GLA and their linear-relation semantics.

diagram	type	interpretation
	$2 \rightarrow 1$	$\{((x_1, x_2), y) \mid y = x_1 + x_2\}$
	$0 \rightarrow 1$	$\{(*, 0)\}$
	$1 \rightarrow 2$	$\{(x, (y_1, y_2)) \mid y_1 = x = y_2\}$
	$1 \rightarrow 0$	$\{(x, *) \mid x \in \mathbb{K}\}$
	$1 \rightarrow 1$	$\{(x, y) \mid y = kx\}, \text{ for } k \in \mathbb{K}$

The white node is addition and the black node is equality. Their reflected forms have the following readings:

- white comultiplication $1 \rightarrow 2$ imposes $x = y_1 + y_2$;
- white counit $1 \rightarrow 0$ imposes $x = 0$;
- black multiplication $2 \rightarrow 1$ imposes $x_1 = x_2 = y$;
- black unit $0 \rightarrow 1$ allows an arbitrary value of y ;
- a reflected scalar imposes $x = ky$.

Thus direction distinguishes solving an equation forward from treating the same equation as a constraint. When $k \neq 0$, a reflected k -box is equivalent to a forward k^{-1} -box; when $k = 0$, its semantics is the non-functional constraint $x = 0$ with arbitrary output.

A.6.1 The equations of GLA

The equational theory can be organized into four layers.

- (i) The addition node and zero form a commutative monoid, while copy and discard form a cocommutative comonoid. Reflecting these equations gives the corresponding structures in the opposite direction.
- (ii) Each color, together with its reflection, is a special commutative Frobenius algebra. The spider theorem therefore applies separately to connected monochromatic regions.
- (iii) The two colors interact by the bialgebra and Hopf laws. These laws express that copying preserves addition and zero, and that additive inverses cancel. The scalar -1 is the antipode.
- (iv) Scalar boxes obey the arithmetic of $\mathbb{K}$: consecutive boxes multiply, parallel copied-and-added boxes add, 1 is the identity, and scalars commute with the appropriate copying and addition nodes.

Every equation is sound because both sides denote the same linear subspace. For example, copying after addition and adding copied components both denote

$$\{(x_1, x_2), (y_1, y_2) \mid y_1 = x_1 + x_2 = y_2\}.$$

The equations do more than preserve semantics: they characterize it.

Theorem A.1 (Presentation theorem for GLA). *For a field $\mathbb{K}$, the standard interpretation extends to an isomorphism of PROPs*

$$\llbracket - \rrbracket : \mathbf{GLA}_{\mathbb{K}} \xrightarrow{\cong} \mathbf{LinRel}_{\mathbb{K}}.$$

Consequently, GLA is sound, complete, and expressive for linear relations [BSZ17; PRS22].

Proof idea. The matrix fragment generated by addition, zero, copy, discard, and scalars represents every matrix over $\mathbb{K}$. Reflection represents converse relations. Since every linear relation has a cospan presentation $Ex = Fy$, every relation is represented by a diagram consisting of a matrix E followed by the converse of a matrix F ; this proves expressivity. The interacting Hopf and Frobenius equations rewrite arbitrary diagrams to such a form. Equality of the resulting linear subspaces is then reducible to ordinary matrix equivalence, whose elementary operations are derivable in the calculus. This yields completeness; soundness follows by checking the relations assigned to the generators and equations. $\square$

The theorem explains why GLA is useful in the main text. A diagram is not merely a picture of a chosen matrix calculation: it denotes the entire solution relation of a linear system. Sequential composition existentially eliminates internal variables, tensor combines independent systems,

and reflection exchanges a relation with its converse. Diagram rewriting is therefore a complete algebra of linear systems.

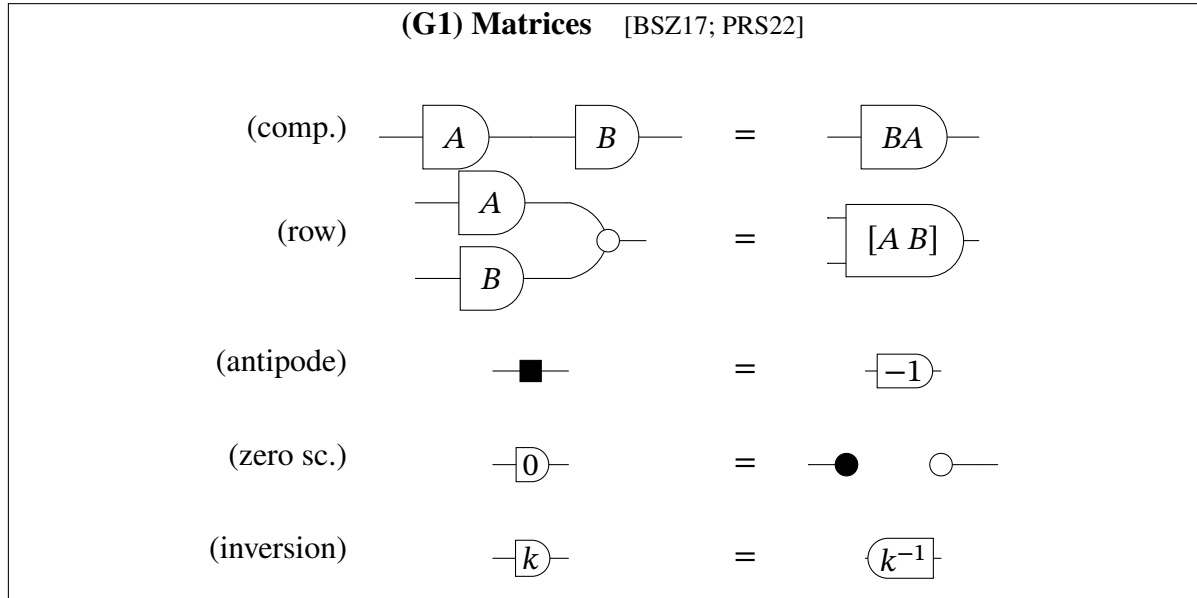
A.6.2 Notation and derived rules used in this thesis

A matrix box $A : m \rightarrow n$ abbreviates any GLA diagram whose interpretation is $y = Ax$. Its reflected box denotes the converse relation. For compatible matrices, connecting boxes gives matrix multiplication, while copying inputs, applying two matrices, and adding their outputs gives the corresponding block matrix. A general relation box may be replaced by its cospan form

$$x R y \iff Ex = Fy.$$

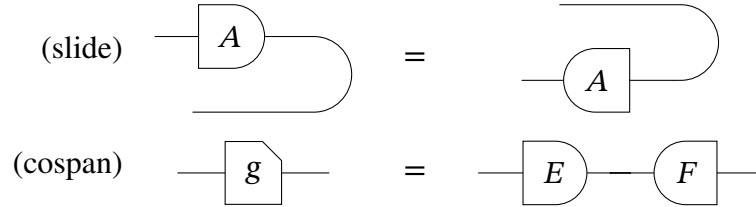
Cups and caps bend wires, transposing a diagram from the functional fragment into its relational converse. Finally, connected black Frobenius diagrams contract by the spider theorem.

The particular derived equalities invoked by the proofs in Chapters 3 and 4 are collected below. They are grouped as (G1) matrix calculus, (G2) transposition and cospan form, (G3) bending, and (G4) the black spider laws. All follow from the GLA equations summarized above and are included here so that later diagrammatic derivations can be checked without expanding every matrix box into scalar generators.

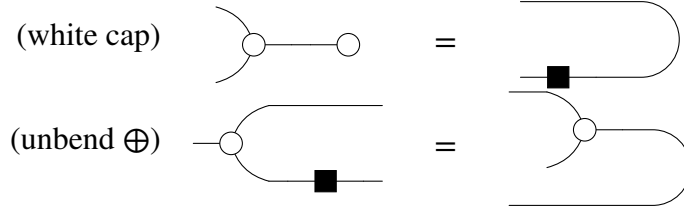


(G2) Transposition and cospan form [BSZ17]

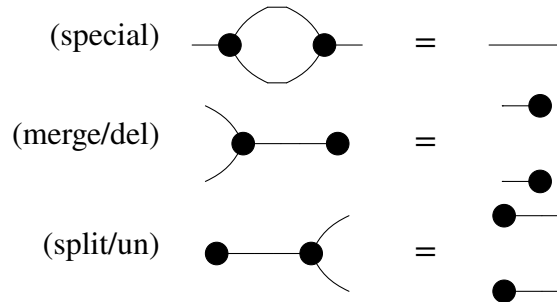
A diagram may be slid along a cap, exchanging it for its transpose; on GLA diagrams the transpose is the converse relation, so a matrix becomes a comatrix.

**(G3) Bending** [BSZ17; Bon+19]

The white cap is the black cap with an antipode on one leg; this is what turns an equation into a pair of bendable wires.

**(G4) The black structure is a spider** [BSZ17]

Any connected network of black nodes with a inputs and b outputs equals the single node $cc_a; cp_b$; the two laws below are the cases used here.



To pass from linear to affine relations one adjoins a constant point $1 : 0 \rightarrow 1$; this gives Graphical Affine Algebra (GAA) [Bon+19]. The later calculi in this thesis extend that affine base with order and optimization generators. Their new equations are studied in the main chapters, whereas the categorical and linear rules on which they rely are exactly those established in this appendix.